



管理员指南

Browser Policy Manager 0.9.3

目录

系统要求和交付范围

- 浏览器策略管理器的最低系统要求
- 评估当前单节点源准备情况
- 记录高可用和生产延期边界

在开源操作系统上从源代码部署浏览器策略管理器

- 准备开源系统主机以进行源部署
- 设置开源系统源签出
- 配置开源系统源运行时设置
- 验证开源系统源部署
- 在乌班图长期支持版上从源代码安装浏览器策略管理器
- 在德比安上从源代码安装浏览器策略管理器
- 在费多拉上从源代码安装浏览器策略管理器
- 在薄荷系统上从源安装浏览器策略管理器
- 在曼扎罗稳定分支上从源代码安装浏览器策略管理器

通过视窗子系统从源代码部署浏览器策略管理器

- 准备视窗系统以进行子系统源部署
- 在子系统内设置浏览器策略管理器源签出
- 配置子系统运行时和浏览器访问
- 通过子系统验证浏览器策略管理器源部署

运行和更新源代码部署

- 查看开发与运维配置源
- 规划存储、日志和备份证据
- 检查网络、跨源资源共享和安全限制
- 记录源操作边界
- 在从源更新之前准备证据
- 刷新源修订版和依赖项
- 运行迁移和源更新检查
- 验证更新并应用回滚停止条件
- 检查浏览器策略管理器运行状况和准备情况

通过编程接口集成浏览器策略管理器

- 集成受众和编程接口边界
- 支持的集成模式
- 编程接口约定

- 当前编程接口限制和安全约束
- 重用开发与运维编程接口示例和环境模板
- 同步配置档案生命周期数据
- 存档、恢复、永久删除或重置配置档案
- 通过编程接口导入火狐浏览器策略文件
- 通过编程接口导出规范火狐浏览器策略文件
- 通过编程接口验证候选火狐浏览器策略文件

运行生命周期 workflow 和从故障中恢复

- 运行拉取、比较和更新控制产品场景
- 运行导入、审核、导出和合规性移交场景
- 从外部控制产品提取并检查浏览器策略管理器配置档案
- 使用预期修订字段验证并应用配置档案更新
- 运行导入、审核、导出和合规性元数据移交
- 通过浏览器策略管理器运行状况和就绪状态启动控制产品
- 从集成失败中恢复并记录审计证据

排除故障并评估生产就绪情况

- 对启动失败和无法访问的探测器进行故障排除
- 排查架构缓存和编程接口验证错误
- 火狐浏览器导入和导出失败故障排除
- 解决数据库和存储问题
- 子系统网络和过时依赖项故障排除
- 排查不可用的文档门户
- 查看网络暴露和代理准备情况问题
- 计划监控输入、备份证据和更新窗口

浏览器策略管理器文档助手

- 在当前版本中使用文档助手
- 维护当前版本的文档助手边界

系统要求和交付范围

浏览器策略管理器的最低系统要求

这些最低要求适用于 BPM 0.9.3 支持的源码运行流程。

浏览器策略管理器基础环境

请通过受支持的 Linux x86_64 源码流程运行 BPM：Ubuntu 26.04 LTS、Debian 13.5、Fedora Linux 44、Linux Mint 22.3 或当前稳定的 Manjaro 分支。Windows 10 和 Windows 11 仅作为该 Linux 流程的 WSL 2 主机受支持。BPM 不提供原生 Windows 服务或安装程序。源码检出、虚拟环境、数据库、日志和文档构建需要 CPython 3.14、两个 CPU 核、4 GiB 内存和 4 GB 可写可用磁盘空间。

浏览器、网络 and 访问

使用启用 JavaScript 和 HTTPS 的当前桌面浏览器操作 BPM Web 界面和文档门户。受支持的 Firefox 架构列表说明策略数据兼容性，并不要求在 Firefox 中运行 BPM UI。只有安装操作系统软件包或源码依赖项时才需要出站 HTTPS 和 sudo。运行进程需要可写入 BPM 检出目录和已配置数据目录的普通 Linux 用户；不要仅为运行应用程序而使用 root。

0.9.3 中的文档助手

文档助手聊天窗口可作为读者界面使用，但 0.9.3 不交付本地模型、RAG 索引、嵌入、生成式回答、引用或外部来源访问。因此，除 BPM 基础流程外，它不增加 CPU、内存、磁盘、GPU、NPU、云账户或网络要求。

每个已提交的问题都会显示本地化的本地模型训练提示。请使用独立的确定性文档搜索获取当前产品信息。

评估当前单节点源准备情况

确定当前源运行的 BPM 实例是否已准备好进行受控单节点操作，并列出支持该决策的证据。

Linux 或 Windows 10/11 通过 WSL 源部署已完成，依赖项已安装，并且预期的 BPM_DATABASE_URL、BPM_DOCUMENTATION_SITE_DIR 和 BPM_SCHEMA_CACHE_DIR 值已知。

BPM 可以为当前状态单节点源操作做好准备。此评估不是生产强化证书、HA设计或打包服务验收。

1. 记录准确的源和运行时配置。

捕获 `git rev-parse --short HEAD`、BPM 版本、BPM_HOST、BPM_PORT、BPM_DATABASE_URL、BPM_DOCUMENTATION_SITE_DIR、BPM_SCHEMA_CACHE_DIR，以及流程是使用 `make dev` 还是直接拥有的 `uvicorn app.main:app` 命令启动。

2. 从用户将使用的同一网络边界运行活动性和就绪性检查。

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

将结果记录为 GET /health 和 GET /health/ready 证据。通过探针仅证明源进程的活跃性和就绪性。

3. 在共享实例之前检查数据和文档证据。

确认 data/bpm.db 是目标数据库，至少一个配置文件可以导出为 policies.json，并且 /help/ 打开构建的文档工件或记录的不可用状态。

4. 将实例分类为当前状态源操作。

警告：

不要将实例标记为打包、强化、负载平衡或高可用。如果外部控制添加这些属性，请将它们记录为外部责任，并将 BPM 证据限制为单节点源操作。

准备说明说明了今天可以从源头安全地完成哪些操作、哪些检查已通过以及哪些决策必须等待分发和运行时支持。

在将实例暴露到本地管理员工作站或受控实验室主机之外之前，请使用网络暴露和代理就绪主题。

记录高可用和生产延期边界

说明目前缺少哪些生产、HA、反向代理、恢复和服务管理功能，以及可以为以后的设计工作做好哪些准备。

当前源运行实例存在准备情况、网络暴露、备份/导出、监控和更新窗口注释。

该主题记录当前单节点源操作与未来生产架构之间的边界。它故意明确了计划中但尚未实施的内容。

1. 命名支持的当前形状。

写入 BPM 管理员/DevOps 文档支持源部署、手动配置、运行状况/就绪检查、API 示例、手动备份证据、文档重建检查和单节点更新窗口。

2. 命名缺少的服务和代理功能。

列出 BPM 源文档中缺少的打包服务、本机 Linux 服务单元、Windows 服务、官方反向代理配方、TLS/代理配方、托管机密、身份验证网关配置和生产强化。

3. 名称缺少 HA 和升级功能。

将支持的 HA 集群、多节点协调、负载均衡器会话关联指导、共享存储锁定、零停机迁移、滚动升级、自动故障转移和官方恢复自动化列为未来工作，而不是当前行为。

4. 将未来需求转化为设计输入。

警告：

不要将多个源节点放置在负载均衡器后面并调用支持的结果HA。捕获所需的 RTO/RPO、备份验证、TLS 所有权、秘密所有权、日志记录、监控和升级窗口期望，作为后续分发/运行时任务的输入。

边界记录可防止不受支持的生产、HA 和反向代理声明，同时保留未来版本的有用准备工作。

在手动文档 QA 期间以及在批准未来安装程序、服务、反向代理和 HA 积压项目时使用此记录。

在开源操作系统上从源代码部署浏览器策略管理器

准备开源系统主机以进行源部署

在从存储库签出安装 BPM 之前，确认受支持的源部署边界、主机先决条件和延迟的生产功能。

使用可以安装 Python 3.14+、Git、构建工具和 curl 的 Linux 主机。此操作手册假定源签出，而不是打包的安装程序。

BPM 可以从源代码运行以进行开发、审查和受控管理评估。此版本中未提供打包安装程序、本机服务单元、官方 systemd 配方、反向代理和 TLS 配方、HA 集群、托管机密和生产强化，并且不得将其呈现为受支持的部署行为。

1. 确定部署目的并记录这是基于源的 Linux 部署。

预期结果：操作说明表明 BPM 是从 Git 签出启动的，并且特定于发行版的安装程序被推迟。

2. 安装或验证主机先决条件。

所需工具包括 Git、Python 3.14+、pip、make、curl 和可以运行 `source .venv/bin/activate` 的 shell。存储库命令使用本地虚拟环境，不需要全局 BPM 包。

3. 创建一个与生产数据和机密分开的工作目录。

将存储库签出、.venv 和本地 SQLite 数据保留在受控路径下。不要在文档示例或提交的文件中存储真正的秘密。

4. 在继续之前确认有意未涵盖的内容。

警告：

此操作手册不提供打包安装程序、本机服务、支持的 systemd 单元、反向代理配方、TLS 终止、HA 拓扑、托管机密存储、备份自动化或生产强化清单。

主机和操作假设已准备好支持受支持的 BPM 源签出工作流程。

在配置运行时变量之前继续 Linux 源签出主题。

设置开源系统源签出

克隆 BPM 存储库、创建 Python 虚拟环境、安装开发依赖项并准备本地数据库架构。

Linux 主机已准备就绪，管理员可以访问版本 0.9.1 的 BPM 存储库 URL 和目标分支或标记。

存储库 README 定义支持的源设置序列。使用存储库本地命令，以便以后的故障排除可以将环境与已提交的脚本进行比较。

1. 克隆存储库并进入结账。

```
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

将占位符替换为已批准的内部存储库 URL 和发布参考。

2. 创建并激活本地Python环境。

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

虚拟环境保留在结账内且未提交。

3. 从结账中安装 BPM 源依赖项。

```
pip install -e ".[dev]"
```

可编辑安装使运行时代码和测试与当前源代码树保持一致。

4. 对选定的运行时数据库应用数据库迁移。

```
alembic upgrade head
```

警告：

仅针对预期的 BPM 数据库运行迁移。对于默认源工作流程，这是本地 SQLite 数据库，而不是生产服务数据库。

签出具有激活的 Python 环境、安装的依赖项和迁移的数据库架构。

在启动应用程序流程之前配置运行时环境。

配置开源系统源运行时设置

设置源运行 BPM 流程使用的数据库 URL、主机、端口、重新加载模式、CORS 和文档站点路径。

已安装签出依赖项，并已为您计划使用的数据库应用迁移。

BPM 读取带有 BPM_ 前缀的环境变量，并且还支持存储库本地 .env 文件。默认数据库 URL 为 sqlite+aiosqlite:///./data/bpm.db，其将数据存储在 data/bpm.db 中。

1. 创建本地数据目录并定义数据库 URL。

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
```

对于可重复的源运行，相同的值可能存储在未提交的本地 .env 文件中。

2. 设置本地源进程的网络和重新加载值。

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

使用环回进行本地验证。绑定到共享接口是此源运行手册边界之外的操作决策。

3. 查看 CORS 和文档站点设置。

源默认值包括 BPM_ENABLE_CORS、BPM_CORS_ALLOW_ORIGINS 和 BPM_DOCUMENTATION_SITE_DIR。保留允许的 CORS 仅用于受控本地评估，并在已安装的静态文档站点可用时将 BPM_DOCUMENTATION_SITE_DIR 指向该站点。

4. 从存储库脚本启动 BPM。

```
make dev
```

Makefile 运行 `uvicorn app.main:app --reload --port 8000`。保持前台终端打开，以便启动消息和回溯可见。

BPM 从具有显式本地运行时设置的源运行。

验证健康状况、准备情况、产品 UI 以及首次行政烟雾检查。

验证开源系统源部署

使用运行状况探测、Web UI、日志和狭义测试命令来确认源运行的 BPM 实例可用。

BPM 流程从存储库签出运行，具有预期的 BPM_DATABASE_URL、主机、端口和文档站点设置。

当前应用程序在 /health 处公开活动状态，在 /health/ready 处公开就绪状态。这些探测器仅确认正在运行的进程响应；它们不能证明生产强化、Firefox 强制执行、备份准备情况或 HA 行为。

1. 检查 Linux 主机的活跃度和准备情况。

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

预期响应为 {"status":"ok"} 和 {"status":"ready","ready":true}。

2. 打开产品 UI 并确认库路由有响应。

使用 `http://127.0.0.1:8000/profiles`。该路由应加载 BPM 界面或显示有意的应用程序状态，而不是服务器回溯。

3. 检查前台应用程序日志。

检查运行 `make dev` 或 `uvicorn app.main:app --reload --port 8000` 的终端。记录启动错误、迁移错误、端口冲突和文档站点警告，而无需将机密复制到票证中。

4. 记录验证结果的支持边界。

警告：

Health 探测、UI 路由和前台日志通过，并不能证明生产就绪、反向代理行为、托管备份或 Firefox 策略在运行时得到实施。

管理员有证据表明 Linux 源部署已启动、应答运行状况/就绪探测、为 UI 路由提供服务，并且没有立即的前台日志阻止程序。

如果验证失败，请将命令、环境变量名称、探测响应和相关日志行放在一起以进行故障排除。请勿将此操作手册扩展为延迟安装程序或生产拓扑说明。

在乌班图长期支持版上从源代码安装浏览器策略管理器

使用发行版 Python 3.14 完整安装 BPM 0.9.1。

需要 Ubuntu 26.04 LTS、sudo、HTTPS 和已批准的 BPM Git 引用。

此过程用于受控的单节点源代码运行，不安装系统服务。

1. 检查 Ubuntu 并安装必备软件。

```
. /etc/os-release
test "$ID" = "ubuntu" && test "$VERSION_ID" = "26.04"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make python3.14 python3.14-dev
python3.14-venv
export BPM_PYTHON=/usr/bin/python3.14
"$BPM_PYTHON" --version
```

2. 检出已批准的引用。

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. 安装 BPM 并迁移数据库。

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. 以前台方式启动 BPM。

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. 检查探针、界面和日志。

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. 安全停止。

失败时按 Ctrl+C，保留输出，不要运行 `alembic downgrade`。

迁移和文档构建完成，探针与配置文件库正常响应。

该命令序列已在干净的 Ubuntu 26.04 用户空间容器中通过验证。此证据不验证原生启动、内核、systemd、桌面、硬件、主机防火墙或生产部署。

在德比安上从源代码安装浏览器策略管理器

使用校验和验证的用户级 CPython 3.14.6 完整安装。

需要 Debian 13.5、sudo、HTTPS、4 GB 可用空间和已批准的引用。

不替换系统 Python 3.13；Python 3.14.6 安装到 `$HOME/.local`。

1. 安装编译必备软件。

```
. /etc/os-release
test "$ID" = "debian" && test "$VERSION_ID" = "13"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlib1g-dev
```

2. 编译已验证的 CPython 3.14.6。

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. 检出 BPM。

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. 安装并迁移 BPM。

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

```
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. 启动 BPM。

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. 检查结果。

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. 安全停止。

保留校验和与日志；诊断前不要降级数据库或删除 Python。

Python、BPM、迁移、文档和探针均已准备就绪。

该命令序列已在干净的 Debian 13.5 用户空间容器中通过验证。CPython 报告可选模块 `_dbm` 和 `_zstd` 不可用，但完整的 BPM 工作流程仍然通过。此证据不验证原生启动、内核、systemd、桌面、硬件、主机防火墙或生产部署。

在费多拉上从源代码安装浏览器策略管理器

使用 Fedora Python 3.14 完整安装 BPM。

需要 Fedora 44、sudo、HTTPS 和已批准的引用。

这是单节点源代码运行，不是 systemd 或生产方案。

1. 安装必备软件。

```
. /etc/os-release
test "$ID" = "fedora" && test "$VERSION_ID" = "44"
sudo dnf install -y ca-certificates curl gcc gcc-c++ git make python3 python3-devel
export BPM_PYTHON=/usr/bin/python3
"$BPM_PYTHON" --version
```

2. 检出 BPM。

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. 安装并迁移 BPM。

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. 启动 BPM。

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. 检查结果。

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. 安全停止。

保留 DNF、pip、迁移、构建和日志输出；不要降级数据库。

BPM 已迁移，文档已构建，探针正常响应。

该命令序列已在干净的 Fedora 44 用户空间容器中通过验证。此证据不验证原生启动、内核、systemd、桌面、硬件、主机防火墙或生产部署。

在薄荷系统上从源安装浏览器策略管理器

通过在当前用户下安装经过校验和验证的 CPython 3.14.6，在 Linux Mint 22.3 上运行完整的 BPM 0.9.1 源安装序列。

使用具有 sudo 访问权限的 Linux Mint 22.3 (Zena)、出站 HTTPS、至少 4 个 GB 可用构建空间以及经批准的 BPM 0.9.1 Git 参考。

Linux Mint 22.3 使用 Ubuntu Noble 软件包基础，其默认 Python 低于 BPM 3.14 的要求。系统 Python 没有被替换。

1. 验证 Mint 并安装 CPython 构建先决条件。

```
. /etc/os-release
test "$ID" = "linuxmint" && test "$VERSION_ID" = "22.3"
sudo apt-get update
sudo apt-get install -y build-essential ca-certificates curl git make libbz2-dev libffi-dev libgdbm-
dev liblzma-dev libncurses-dev libreadline-dev libsqlite3-dev libssl-dev tk-dev uuid-dev xz-utils
zlibg-dev
```

2. 构建经过验证的 CPython 3.14.6 源版本。

```
export PYTHON_VERSION="3.14.6"
export PYTHON_SHA256="143b1dddefaec3bd2e21e3b839b34a2b7fb9842272883c576420d605e9f30c63"
export PYTHON_PREFIX="$HOME/.local/python-$PYTHON_VERSION"
mkdir -p "$HOME/src" && cd "$HOME/src"
curl -fLO "https://www.python.org/ftp/python/$PYTHON_VERSION/Python-$PYTHON_VERSION.tar.xz"
printf '%s  %s\n' "$PYTHON_SHA256" "Python-$PYTHON_VERSION.tar.xz" | sha256sum -c -
tar -xf "Python-$PYTHON_VERSION.tar.xz"
cd "Python-$PYTHON_VERSION"
./configure --prefix="$PYTHON_PREFIX" --with-ensurepip=install
make -j"$(nproc)"
make altinstall
export BPM_PYTHON="$PYTHON_PREFIX/bin/python3.14"
"$BPM_PYTHON" --version
```

3. 查看批准的 BPM 源参考。

```
cd "$HOME"
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd browser-policy-manager
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

4. 创建环境、安装 BPM 并迁移。

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

5. 在前台启动 BPM。

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

6. 从第二个终端验证探针和 UI。

```
cd "$HOME/browser-policy-manager"
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

7. 应用停止和回滚边界。

失败时，按 Ctrl+C，保留日志和校验和输出，并且不运行 `alembic downgrade`。保留用户拥有的 Python 前缀并签出直至诊断。

Linux Mint 22.3 主机具有用户拥有的 Python 3.14.6 运行时、迁移的 BPM 源签出、构建文档、传递探针、响应 UI 和前台日志。

该命令序列已在干净的 Linux Mint 22.3 用户空间容器中通过验证。CPython 报告可选模块 `_dbm` 和 `_zstd` 不可用，但完整的 BPM 工作流程仍然通过。此证据不验证原生启动、内核、systemd、桌面、硬件、主机防火墙或生产部署。

在曼扎罗稳定分支上从源代码安装浏览器策略管理器

完整更新稳定分支并检查 Python 3.14 后安装 BPM。

使用不早于 2026-06-26 更新的稳定分支，并具备 sudo、HTTPS 和已批准的引用。

Manjaro 为滚动更新；必须记录分支和完整更新状态。

1. 更新系统并安装必备软件。

```
. /etc/os-release
test "$ID" = "manjaro"
test "$(pacman-mirrors -G)" = "stable"
sudo pacman -Syu --needed --noconfirm base-devel ca-certificates curl git python python-pip
pacman-mirrors -G
uname -r
export BPM_PYTHON=/usr/bin/python
"$BPM_PYTHON" --version
```

2. 检出 BPM。

```
export BPM_REPOSITORY_URL="https://github.com/Goudron/browser-policy-manager.git"
export BPM_REF="<approved-0.9.1-ref>"
test "$BPM_REF" != "<approved-0.9.1-ref>"
cd "$HOME"
git clone "$BPM_REPOSITORY_URL" browser-policy-manager
cd "$HOME/browser-policy-manager"
git checkout --detach "$BPM_REF"
git status --short
git rev-parse --short HEAD
```

3. 安装并迁移 BPM。

```
"$BPM_PYTHON" -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
python --version
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

4. 启动 BPM。

```
export BPM_HOST="127.0.0.1"
export BPM_PORT="8000"
export BPM_RELOAD="true"
make dev
```

5. 检查结果。

```
cd browser-policy-manager
source .venv/bin/activate
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
curl -fsS -o /dev/null http://127.0.0.1:8000/profiles
```

6. 安全停止。

保留分支、包和日志信息；不要部分降级包或数据库。

BPM 和文档已就绪，探针正常响应。

该命令序列已在记录分支和完整软件包状态后，于干净的 Manjaro 稳定分支用户空间容器中通过验证。此证据不验证原生启动、内核、systemd、桌面、硬件、主机防火墙或生产部署。

通过视窗子系统从源代码部署浏览器策略管理器

准备视窗系统以进行子系统源部署

启用 WSL，选择受支持的 Linux 发行版假设，并确认 BPM 将在 Linux 内部运行，而不是作为本机 Windows 服务运行。

使用具有启用 WSL 2 权限的 Windows 10 或 Windows 11，安装 Linux 发行版（例如 Ubuntu LTS），并在 PowerShell 和 WSL shell 中运行命令。

此操作手册记录了通过 WSL 进行 BPM 源部署。支持的应用程序流程仍然是 WSL 内部的 Linux 源码工作流程；此版本中不提供本机 Windows 安装程序、Windows 服务、MSI 软件包、计划任务、IIS 托管和生产 Windows 服务管理。

1. 确认来自 PowerShell 的 WSL 可用性。

```
wsl --status  
wsl --list --verbose
```

预期结果：WSL 已安装，WSL 2 可用，并且所选的 Linux 发行版可见。

2. 安装 WSL 和经批准的 Linux 发行版（如果缺少）。

```
wsl --install  
wsl --install -d Ubuntu
```

使用您的组织批准的发行版名称。如果 Windows 请求，请重新启动，然后在发行版首次启动时创建 Linux 用户。

3. 记录分布和文件系统边界。

将 BPM 放置在 WSL Linux 文件系统下，例如 /home/<linux-user>/browser-policy-manager。避免从 /mnt/c 运行签出，因为 Windows 文件系统语义和防病毒扫描可能会使 Python 环境、文件监视和权限不可靠。

4. 确认此 Windows 路径尚不支持什么。

警告：

这不是本机 Windows 安装。操作手册不提供 MSI/EXE 打包、Windows 服务、服务帐户模型、事件查看器集成、Windows 防火墙策略、反向代理/TLS 配方、HA 拓扑、托管机密、备份自动化或生产强化。

Windows 和 WSL 已准备好进行基于 Linux 的 BPM 源部署，具有清晰的平台和文件系统边界。

继续打开 WSL 分发 shell 并在 Linux 主目录下设置源签出。

在子系统内设置浏览器策略管理器源签出

在 WSL 中安装 Linux 先决条件，将 BPM 克隆到 Linux 文件系统中，创建虚拟环境，安装依赖项并应用迁移。

安装并打开 WSL 2 Linux 发行版，并且管理员拥有批准的 BPM 存储库 URL 和 0.9.1 分支或标签。

本主题中的所有命令都在 WSL Linux shell 内运行，除非该步骤明确指出 PowerShell。这使设置与 Linux 源部署操作手册保持一致，并避免本机 Windows 路径歧义。

1. 在 WSL 发行版中安装 Linux 必备组件。

```
sudo apt-get update
sudo apt-get install -y git make curl python3 python3-venv python3-pip
python3 --version
```

使用为您所选的 WSL 发行版批准的包管理器和包名称。

2. 将 BPM 克隆到 Linux 主文件系统中。

```
cd ~
git clone <repository-url> browser-policy-manager
cd browser-policy-manager
git checkout <approved-0.9.1-ref>
```

不要将受支持的 WSL 源工作流的工作签出放在 /mnt/c 下。

3. 创建并激活本地 Python 环境。

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

如果发行版仅将 Python 公开为 python3，请使用 `python3 -m venv .venv` 创建环境，然后使用激活的环境。

4. 在 WSL 结帐内应用数据库迁移。

```
mkdir -p data
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

警告：

SQLite 文件是 WSL 内的 Linux 文件。BPM 运行时，请勿从 Windows 工具编辑或移动 data/bpm.db。

WSL 签出具有 Linux 先决条件、存储库本地虚拟环境、已安装的依赖项和迁移的本地数据库。

在启动 BPM 之前配置运行时变量和浏览器访问。

配置子系统运行时和浏览器访问

在 WSL 内设置 BPM 运行时变量，选择安全绑定地址，启动源进程，并识别来自 Windows 的浏览器 URL。

BPM 签出、虚拟环境、依赖项和迁移已在 WSL Linux 文件系统中准备就绪。

BPM 从 Linux 进程读取 BPM_ 环境变量。现代 WSL 通常会将 localhost 转发到 Windows，但当浏览器访问或防火墙行为不清楚时，可以检查 WSL IP。

1. 在 WSL shell 中设置本地运行时变量。

```
source .venv/bin/activate
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
export BPM_HOST="0.0.0.0"
export BPM_PORT="8000"
export BPM_RELOAD="true"
```

BPM_HOST="0.0.0.0" 允许 Windows 浏览器通过 localhost 转发到达 WSL 进程。仅当浏览器也在 WSL 内或转发已验证时才使用 127.0.0.1。

2. 从 WSL 结帐启动 BPM。

```
make dev
```

Makefile 启动 uvicorn app.main:app --reload --port 8000。保持 WSL 终端打开以获取启动消息、回溯和文件观察器输出。

3. 识别 Windows 浏览器 URL。

```
hostname -I
```

首先从 Windows 尝试 <http://localhost:8000/profiles>。如果本地主机转发不可用，请使用 `hostname -I` 报告的第一个 WSL IP，例如 <http://<wsl-ip>:8000/profiles>。

4. 保持安全边界明确。

警告：

向其他主机打开 WSL 端口、更改 Windows 防火墙规则、添加 TLS 或在 BPM 前面放置反向代理不属于此 WSL 源部署主题，必须等待生产就绪指导。

BPM 源进程在 WSL 内运行，并具有已识别的 Windows 浏览器访问路径。

验证运行状况、准备情况、UI 访问和 WSL 特定的故障模式。

通过子系统验证浏览器策略管理器源部署

检查来自 WSL 和 Windows 的探测，确认 UI 路由，检查日志，并对常见的 WSL 网络和文件系统问题进行故障排除。

BPM 正在从 WSL 签出运行，具有预期的 BPM_DATABASE_URL、主机、端口和文档站点设置。

验证必须证明 Linux 进程响应和 Windows 浏览器访问路径。它不认证本机 Windows 安装、生产强化、反向代理行为、备份准备情况或 Firefox 运行时强制。

1. 检查 WSL 内的运行状况和准备情况。

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

预期响应为 {"status":"ok"} 和 {"status":"ready","ready":true}。

2. 从 Windows PowerShell 检查相同的探针。

```
Invoke-WebRequest http://localhost:8000/health
Invoke-WebRequest http://localhost:8000/health/ready
```

如果 localhost 不起作用，请使用 hostname -I 中的 WSL IP 重试。记录成功的地址。

3. 从 Windows 浏览器打开 UI 路由。

使用 http://localhost:8000/profiles 或 http://<wsl-ip>:8000/profiles。库路由应该加载 BPM 接口或有意的空/错误状态，而不是回溯。

4. 在更改 BPM 代码之前对 WSL 特定的故障进行故障排除。

对于连接失败，请确认 make dev 仍在运行，检查 BPM_HOST，运行 wsl --list --verbose，并在重新启动分发后刷新 WSL IP。对于文件监视速度慢或权限错误，请将结帐从 /mnt/c 移至 /home/<linux-user>。请在环境路径和网络路由稳定后运行。

警告：

通过 WSL 通过运行状况检查并不能证明生产准备情况、Windows 服务行为、托管备份或 HA 支持。

管理员有证据表明 BPM 在 WSL 内部启动、响应探测、可从 Windows 访问，并且没有直接的 WSL 网络或文件系统阻止程序。

将 PowerShell 输出、WSL shell 命令、BPM_* 变量名称、探测响应和前台日志放在一起，以便在不复制机密的情况下进行故障排除。

运行和更新源代码部署

查看开发与运维配置源

识别当前的 BPM 配置源，记录有效的运行时值，并将普通设置与不受支持的秘密管理期望分开。

BPM 从 Linux 上的源代码或通过 WSL 部署，管理员可以检查 shell 环境、可选的 .env 文件和源代码签出。

BPM 从带有 BPM_ 前缀的环境变量和存储库根 .env 文件加载设置。配置模型位于已记录的默认值中。此版本未定义应用程序秘密存储、凭证库、API 令牌设置、加密密钥设置或生产配置文件。

1. 列出用于此运行的配置源。

记录值是来自 shell 导出、本地 .env 还是已记录的默认值中的默认值。将 .env 保留在结账本地，并且不提交它。

2. 记录影响流程行为的运行时设置。

包括 BPM_APP_NAME、BPM_APP_VERSION、BPM_DEBUG、BPM_HOST、BPM_PORT、BPM_RELOAD、BPM_API_PREFIX、BPM_DEFAULT_LOCALE、BPM_SUPPORTED_LOCALES、BPM_I18N_DIR 和 BPM_DOCUMENTATION_SITE_DIR。默认值是本地源默认值，而不是生产策略。

3. 分别记录数据、模式和网络相关设置。

跟踪 BPM_DATABASE_URL、BPM_DB_ECHO、BPM_ENABLE_CORS、BPM_CORS_ALLOW_ORIGINS、BPM_SCHEMA_BASE_URL、BPM_SCHEMA_CACHE_DIR 和 BPM_SCHEMA_HTTP_TIMEOUT。将数据库 URL 和网络来源视为操作证据，即使它们不包含密码。

4. 对 BPM 配置示例保密。

警告：

BPM 源部署文档不提供身份验证设置、托管机密、静态加密配置、令牌轮换、租户隔离或机密编辑保证。如果您的环境需要这些控件，请将它们记录为外部平台控件而不是 BPM 支持的功能。

部署具有经过审核的配置清单，该配置清单与当前源设置相匹配，并不意味着不支持秘密管理行为。

在将 BPM 暴露到受控本地环境之外之前，使用存储、网络和操作边界主题来检查记录值的影响。

规划存储、日志和备份证据

找到源部署数据库、架构缓存、文档站点、前台日志和手动备份证据，而无需声明托管备份或恢复支持。

部署配置清单包括 `BPM_DATABASE_URL`、`BPM_SCHEMA_CACHE_DIR` 和 `BPM_DOCUMENTATION_SITE_DIR`。

默认源部署将应用程序数据存储在于 `sqlite+aiosqlite:///./data/bpm.db` 的 SQLite 中，对应于存储库根目录下的 `data/bpm.db`。默认情况下，架构缓存文件位于 `app/schemas/mozilla` 下，并且从 `app/documentation/site` 中读取已安装的文档工件（如果存在）。

1. 识别保持操作状态的文件和目录。

记录存储库路径、`data/`、`data/bpm.db`、`app/schemas/mozilla` 和 `app/documentation/site`。在 WSL 中，将这些路径保留在 Linux 文件系统中，而不是在 `/mnt/c` 下。

2. 在危险工作之前定义手动备份证据步骤。

停止前台 BPM 进程，将 `data/bpm.db` 复制到受控备份位置，并记录时间戳、源版本、数据库 URL、文件大小和校验和。BPM 不提供自动备份、时间点恢复或经过测试的恢复编排。

3. 从当前源进程捕获日志。

源运行将启动消息、Uvicorn 访问输出、回溯、迁移错误、端口冲突和文档站点警告写入前台终端、`stdout` 或 `stderr`。此版本中没有记录专用的 BPM 应用程序日志目录、日志轮换策略、请求 ID、审核日志或事件查看器集成。

4. 当配置文件移交很重要时，将文件备份证据与导出证据配对。

使用现有 API 导出示例导出相关 Firefox `policies.json` 文档并存储响应状态、配置文件 ID、修订、架构通道和导出文档哈希。不要将导出证据视为完整的数据库恢复。

管理员知道当前源部署状态位于何处，以及在更新或故障排除之前可以收集哪些手动证据。

在更改源修订或数据库架构之前，请使用未来的从源更新 操作手册，并在备份或还原证据不完整时停止。

检查网络、跨源资源共享和安全限制

在使源运行的 BPM 实例可访问之前，请检查主机、端口、重新加载、运行状况探测、CORS 默认值和安全限制。

源进程可以使用 `make dev` 启动，管理员可以更改 `BPM_HOST`、`BPM_PORT`、`BPM_RELOAD`、`BPM_ENABLE_CORS` 和 `BPM_CORS_ALLOW_ORIGINS`。

当前默认值针对本地源操作进行了优化：`BPM_HOST="0.0.0.0"`、`BPM_PORT="8000"`、`BPM_RELOAD="true"`、`BPM_ENABLE_CORS="true"` 以及相当于 `["*"]` 的许可源。这些默认值不是强化的网络暴露策略。

1. 选择当前环境的最窄绑定地址。

使用 `127.0.0.1` 进行本地 Linux 检查。仅当 WSL 浏览器转发或受控实验室网络需要时才使用 `0.0.0.0`。记录任何非环回绑定的原因。

2. 通过运行状况和就绪探测控制操作访问。

在 API 示例或 UI 烟雾检查之前使用 `GET /health` 和 `GET /health/ready`。成功的探测仅显示进程的活跃性/就绪性；它们不证明数据库备份状态、Firefox 强制执行、身份验证、授权或生产拓扑。

3. 限制任何共享评估的 CORS。

对于本地实验，宽松的 `BPM_CORS_ALLOW_ORIGINS` 可能是可以接受的。对于共享主机，设置显式来源并记录谁拥有浏览器来源。BPM 文档未定义凭据 CORS、CSRF 保护、API 身份验证或租户隔离。

4. 将不受支持的安全控制排除在 BPM 声明之外。

警告：

此版本不提供 TLS 终止、反向代理配方、防火墙策略、速率限制、请求签名、审核日志记录、SSO、OAuth/OIDC 设置、基于角色的访问控制、漏洞扫描或 HA 故障转移。如果您的环境添加了它们，请将它们记录为外部控制。

源运行的 BPM 网络状态被记录为受控的本地或实验室暴露，而不是生产安全架构。

如果探测失败或暴露不清楚，请在继续 API 集成示例之前使用源部署验证主题。

记录源操作边界

创建操作移交注释，说明运行时用户、流程模型、支持的检查和延迟的生产功能。

源运行实例的配置、存储、备份、网络和 CORS 审核已完成。

BPM 管理员/DevOps 文档当前支持源运行操作以进行受控评估。它尚未为每个企业控件提供分发安装程序、本机服务、官方守护进程、生产拓扑或完整的操作运行手册。

1. 记录运行时用户和流程模型。

识别拥有结账、.venv、data/ 以及正在运行的 `make dev` 或 `uvicorn app.main:app --reload --port 8000` 进程的 shell 用户。不要声明受支持的服务帐户、systemd 单元、Windows 服务或重新启动策略，除非您的外部平台提供并拥有它。

2. 记录今天支持的检查。

包括源修订、活动配置值、迁移状态、/health、/health/ready、UI 路由烟雾、API 示例状态、前台日志审查和手动备份证据。将 API 证据链接到可重用的 API 示例，而不是重复端点过程。

3. 将延迟功能记录为阻碍因素或外部责任。

调出打包安装程序、本机 Linux 或 Windows 服务、反向代理和 TLS 配方、HA 集群、托管机密、身份验证配置、备份自动化、恢复编排、监控仪表板、日志轮换和生产强化（BPM 源文档未提供）。

4. 附加面向用户的恢复引用而不移动所有权。

当操作员需要了解 UI 症状时，请链接到面向用户的故障排除，例如产品连接恢复。将部署、API、备份和网络所有权保留在管理员/DevOps 指南中。

移交说明说明了当前源部署可以支持什么、有什么证据证明这一点，以及哪些生产责任仍处于 BPM 之外。

使用此边界注释作为从源更新 操作手册 和后续生产/HA/反向代理文档任务的起点。

在从源更新之前准备证据

在将 BPM 移动到较新的源修订版之前收集源修订版、配置、数据库备份、配置文件导出证据和停止条件。

BPM 正在从 Linux 或 WSL 源签出运行，并且管理员已完成配置、存储、备份和操作边界审查。

从源更新过程会更改代码、依赖项、数据库模式和文档输出期望。BPM 不提供自动回滚、数据库降级保证、托管备份或打包升级工具。将证据收集视为必要的更新前关卡。

1. 停止前台 BPM 进程并记录当前源状态。

```
git status --short
git rev-parse --short HEAD
```

仅当了解工作树状态并将当前源修订记录为回滚引用时才继续。

2. 记录有效配置和运行时路径。

捕获活动的 BPM_DATABASE_URL、BPM_DOCUMENTATION_SITE_DIR、BPM_SCHEMA_CACHE_DIR、BPM_HOST、BPM_PORT，以及值是否来自 shell 导出或 .env。不要将机密复制到更新说明中。

3. 在 BPM 停止时创建手动数据库备份证据。

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

记录时间戳、当前版本、数据库 URL、文件大小和校验和。如果部署不使用 data/bpm.db，请备份从 BPM_DATABASE_URL 解析的路径。

4. 在更改代码之前导出关键配置文件证据。

使用现有的 API 导出示例保存所需的 Firefox policies.json 文档并记录响应状态、配置文件 ID、修订、架构通道和导出文档哈希。

警告：

如果数据库备份或导出证据丢失，请停止更新。不要假设 Git 回滚或 alembic downgrade 是安全的，从而替换丢失的备份证据。

该更新具有经过验证的更改前证据集和已知的先前源参考。

仅在证据集完成并经过审核后才继续刷新源修订和依赖项。

刷新源修订版和依赖项

将签出移至已批准的较新版本并重新安装 Python 环境，而不隐藏依赖项或工作树故障。

存在更新前证据集，记录了先前的修订，并且已知已批准的目标分支、标签或提交。

BPM 源更新是在现有签出中执行的 Git 和 Python 环境操作。该过程不会安装打包发行版、本机服务或生产升级管理器。

1. 获取批准的源参考。

```
git fetch --tags
git status --short
```

如果本地更改意外发生，则停止。保留用户拥有或站点拥有的更改而不是覆盖它们。

2. 移至已批准的较新源版本。

```
git checkout <approved-new-ref>
git rev-parse --short HEAD
```

记录目标修订并将其与批准的变更单或发行说明进行比较。

3. 刷新存储库本地 Python 环境。

```
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -e ".[dev]"
```

使用现有的 .venv 除非更新说明明确要求重建它。在更改数据库之前记录依赖项失败。

4. 在未解决的源或依赖项故障时停止。

警告：

如果签出、依赖项安装或虚拟环境激活失败，请勿运行迁移。仅当数据库未更改时才返回记录的先前版本，然后重新安装先前的依赖项集并重新运行更新前的冒烟检查。

签出和 Python 环境位于已批准的目标修订版上，或者更新在数据库更改之前已停止。

仅在依赖项刷新成功后才运行数据库迁移和文档检查。

运行迁移和源更新检查

在源修订版本更改后，应用数据库迁移并验证更新后的 BPM 源部署。

签出和依赖项已成功刷新，并且在 BPM 停止时收集了数据库备份证据。

更新可能会引入数据库迁移。BPM 记录了使用 `alembic upgrade head` 进行正向迁移；不承诺自动降级或安全的架构回滚。

1. 将数据库迁移应用到目标数据库。

```
export BPM_DATABASE_URL="sqlite+aiosqlite:///./data/bpm.db"
alembic upgrade head
```

使用 `BPM_DATABASE_URL` 的实际部署值。如果迁移输出不清楚或失败，请立即停止。

2. 启动更新后的 BPM 进程并验证当前界面。

检查运行状况和就绪状态，然后打开“库”路由。如果该部署需要文档，请确认 `/help/` 报告的是已就绪的安装工件，而不是缺失或不兼容状态。

3. 在迁移或运行时验证失败时停止。

警告：

迁移运行后，不要认为 Git 签出回滚就足够了。除非已审核的发行说明明确支持该准确修订版本对，否则不要运行 `alembic downgrade`。需要回滚时，请使用更新前的数据库备份作为安全恢复基线。

更新后的源修订版本已完成迁移和运行时验证，或者更新已在指定的故障点停止。

在将实例交还给用户或集成商之前启动 BPM 并运行更新后验证主题。

验证更新并应用回滚停止条件

启动更新的 BPM 流程，验证探针、UI、API 证据和文档状态，然后决定是否接受、停止或从备份恢复。

迁移、源烟雾和文档重建检查已完成或已记录故障点。

更新后验证证明更新后的源运行可用于当前受控环境。它不证明生产准备情况、反向代理行为、HA、托管备份或 Firefox 运行时强制。

1. 启动更新的源进程并检查探测器。

```
make dev
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

预期响应仍然是 {"status":"ok"} 和 {"status":"ready", "ready":true}。

2. 验证产品 UI、API 证据和文档状态。

打开 <http://127.0.0.1:8000/profiles>，运行所需的可重用 API 示例，在需要时确认导出的 policies.json 证据，并检查 /help/ 是否未报告已安装工件的陈旧、不兼容或丢失文档。

3. 仅当记录了每个必需的检查后才接受更新。

记录以前的修订、目标修订、迁移结果、源烟雾结果、文档验证/构建结果、运行状况/就绪响应、UI 烟雾结果、API/导出证据以及任何已知的延期生产控制。

4. 应用回滚或停止条件而不发明自动恢复。

如果在迁移之前发生故障，请返回到先前的源版本并重新安装依赖项。如果迁移后发生故障，请停止 BPM，恢复已验证的更新前数据库备份，返回到先前的源版本，重新安装依赖项，并重复运行状况/UI 烟雾检查。如果备份证据丢失、恢复未经验证或迁移后数据发生更改，请停止并升级，而不是声明回滚成功。

警告：

当探测失败、文档过时、导出意外不同、迁移不确定或回滚证据不完整时，不要将更新标记为完成。

更新要么在完整证据的情况下被接受，在已知的阻止程序处停止，要么从具有明确证据的更新前备份中恢复。

在操作移交记录和未来生产/HA/反向代理规划中使用生成的证据。

检查浏览器策略管理器运行状况和准备情况

使用当前的存活探测和就绪探测作为 API 工作流程之前的轻量级运行状态预检。

设置 `$BPM_BASE_URL`。有关主机设置、服务启动、日志、源更新和操作系统详细信息，请使用源部署管理员指南；此 API 主题仅记录集成状态预检。

此任务涵盖 API-HEALTH-001 和 API-HEALTH-002。BPM 在 `/health` 和 `/health/ready` 处公开了无前缀探针。BPM 不会通过这些探测器公开依赖性诊断；它们没有版本控制，并且不会取代特定于产品的重试或事件策略。

1. 在开始简短的 API 序列之前调用存活探测。

请求示例：GET `$BPM_BASE_URL/health`。响应示例：200 `{"status":"ok"}`。错误示例：当前未声明应用程序错误；传输、网关或流程故障在 BPM 外部处理。

2. 在依赖于应用程序启动完成的操作之前调用就绪探测。

请求示例：GET `$BPM_BASE_URL/health/ready`。响应示例：200 `{"status":"ready","ready":true}`。错误示例：当前未公开任何降级状态或依赖项详细信息主体。

3. 使用就绪性作为门，而不是作为完整的依赖项清单。

当 `ready` 为 `true` 时，集成产品可以继续发现、验证、导入、导出或配置文件生命周期调用。它不得从此响应推断数据库运行状况详细信息、架构缓存详细信息、文档门户状态、队列状态或部署拓扑。

4. 在集成产品中保持轮询并重试决策。

BPM 不声明轮询间隔、重试标头、服务级别目标、断路器合约、请求 ID 或用于探测更改的 Webhook。存储您自己的时间戳、观察到的状态、超时和重试决策。

5. 将 API 握手检查与源部署故障排除分开。

如果探测器无法访问或响应与示例不同，请切换到源部署管理员指南以进行流程、环境、数据库、日志和从源更新检查，而不是将此 API 集成任务扩展为部署说明。

6. 记录与下游 API 操作的运行状态预检结果。

警告：

成功的运行状况或就绪响应不会验证调用者、授权操作、保留配置文件修订、保证以后 API 成功或证明 Firefox 运行时强制执行。将其视为一个小的飞行前信号。

集成产品具有最小且最新的 BPM 运行状态预检，以实现活跃性和就绪性，而不依赖于未记录的部署行为。

成功完成就绪检查后，继续进行 OpenAPI 发现或特定 API 工作流程主题以进行验证、导入/导出或配置文件生命周期操作。

通过编程接口集成浏览器策略管理器

集成受众和编程接口边界

管理员/DevOps 指南中的 API 集成部分适用于使用 BPM 当前编程 API 的外部工具，但不对未支持的产品能力作出保证。

观众

当您构建或审查与 BPM 交换配置文件和 Firefox 策略数据的配置、合规性、编排、库存或控制产品时，请使用本指南。主要读者是 API 集成商、安全审查员、发布工程师和维护人员，他们需要可重复的合同来进行配置文件同步、验证门、策略导入/导出、合规性元数据、库存和运行状况检查。

证据边界

该指南基于生成的 OpenAPI 合约、维护的 API 库存、路线/模型源和重点 API 测试。它记录了 0.9.1 版本中已经公开的 API BPM。它不授权新的端点、安全模型、数据模式、重试语义或兼容性承诺。

支持的消费者角色

支持的使用者是通用角色：配置管理器、合规性扫描器、策略审查门、配置文件库存收集器、部署协调器和操作监视器。他们可以调用 API 清单中列出的服务、配置文件、Firefox 导入/导出、验证和运行状况操作。

不支持的声明

请勿描述特定于产品的连接器、市场集成、认证集成、合作伙伴关系、托管代理或供应商工作流程，除非该连接器实际发货。HTML 路由、静态资产、生成的架构 界面 和下划线前缀的 Python 帮助程序不是稳定的机器集成 API。

数据所有权

BPM 拥有规范化配置文件模型、验证状态、修订、生命周期状态、不透明合规性对象和规范的 Firefox 导出呈现。集成产品拥有自己的调度、授权、审计跟踪、重试策略、异常记录、部署决策以及 BPM 之外的任何运行时验证。

支持的集成模式

BPM 支持配置文件、验证、Firefox 文档、元数据、库存和运行状况工作流程的通用 API 模式。

配置档案同步

集成产品可以列出配置档案、通过 ID 读取配置档案、创建标准化配置档案、修补可变配置档案字段、存档、恢复、永久删除或重置库。同步必须将配置档案 ID 和修订视为 BPM 拥有的状态，并且不得假设跨多个请求的批量操作、名称突变或自动回滚。

验证门

验证门可以提交候选规范 在部署工作流接受之前，将 `policies.json` 记录到受支持的验证端点。门必须检查响应正文以及状态代码，因为验证可以返回成功 HTTP 状态 `ok=false`。

策略导入和导出

集成可以将 Firefox `policies.json` 作为 BPM 配置文件导入，并将保存的 BPM 配置文件导出回规范的 Firefox JSON。不要将规范化的配置文件模型与规范的 Firefox 文档形状互换；导入/导出是这些表示之间的边界。

合规性元数据移交

API 在配置文件创建、更新和导入操作中公开不透明的 `compliance` 对象。集成可以通过 BPM 传递自己的元数据，但 BPM 不会发布该对象的版本化模式，也不会验证外部合规性语义。

库存和健康检查

库存样式的使用者可以列出配置文件、请求同一过滤器上下文的统计信息、读取单个配置文件以及调用服务、活跃度和就绪端点。健康响应有意紧凑，并且不会暴露当前固定字段之外的每个依赖项诊断、文档状态或部署准备情况。

不支持的模式

该指南未记录身份验证、授权、速率限制、幂等密钥、请求 ID、webhooks、批量导出、批量 CRUD、光标分页、产品特定连接器、供应商合作伙伴关系或事务编排，因为 BPM 未公开这些保证。

编程接口约定

在阅读 BPM OpenAPI 合约和构建集成请求时使用这些约定。

基础地址和发现

示例使用 `$BPM_BASE_URL` 作为部署占位符。从 `/openapi.json` 发现当前合约，以及位于 `/docs` 和 `/redoc` 的 FastAPI 架构 界面。未来的产品文档门户是为 `/help/` 保留的，因此不要将 `/docs` 视为 DITA 文档路径。

内容类型

大多数编程操作使用 JSON 请求和响应主体。Firefox 导入接受 `application/json` 和 `multipart/form-data`；不支持的媒体类型可以返回 415。Firefox 导出返回 `application/json`，并且可以在请求下载输出时添加 `Content-Disposition`。

标识符和生命周期

配置文件 ID 是 BPM 拥有的整数路径标识符。存储创建、导入、读取和更新操作后返回的 `id` 和 `revision`。配置文件名称是唯一的，无法通过 `ProfileUpdate` 进行更改。生命周期值包括活动值、存档值和所有值；`include_deleted` 是一个兼容性标志，必须与生命周期一起解释。

模型和模式通道

`ProfileCreate`、`ProfileUpdate` 和 `ProfileRead` 是 BPM 标准化轮廓形状。Firefox 导入/导出使用规范的 `policies.json` 和顶级 `policies`。验证使用 `ValidationRequest` 和支持的模式通道，例如 Firefox ESR 和 Release；简单的策略映射仍然是兼容性行为，而不是首选的外部形状。

过滤、排序和分页

配置文件列表和统计操作支持搜索、模式、验证状态、生命周期和包含-删除过滤器。列表分页是偏移/限制，默认为 `limit=50`，最大为 200 和 `offset=0`。排序使用当前描述的字段，例如 `created_at`、`updated_at`、`name`、`schema_version` 和 `id`，以 `asc` 或 `desc` 顺序。

状态和错误处理

将状态代码和身体形状一起使用。200、201 和 204 是成功系列，但验证可以返回 200 和 `ok=false`。错误主体使用 FastAPI detail，并且可能包含字符串、验证列表或带有 `message`、`error` 或 `issues` 的 BPM 对象。

当前编程接口限制和安全约束

BPM 清楚地记录了当前的限制，因此集成不会推断出未暴露的保证。

安全约束

当前 API 没有身份验证或授权层。CORS 默认为 * 来源，除非部署设置覆盖它。文档不得暗示 API 令牌身份验证、OAuth、基于角色的访问控制、租户隔离或生产授权行为。

版本控制和兼容性

API 没有版本前缀、兼容性策略、弃用标头或媒体类型版本。集成应该固定他们测试的 BPM 版本并观察 `/openapi.json`、API 库存和发行说明，而不是假设有保证的向后兼容性。

并发性、幂等性和重试

只有 PATCH 通过可选的 `expected_revision` 支持乐观并发。创建、导入、删除、恢复、重置和导出不会公开 ETags、前提条件标头、幂等性密钥或重试合约。消费者必须设计自己的安全重试和重名恢复行为。

批量和事务边界

BPM 不公开批量配置文件 CRUD 端点、批量导出端点、游标分页和多操作事务合约。BPM 不会自动回滚失败的多步骤集成；集成产品拥有恢复排序和审计记录。

误差包络限制

错误响应不使用一种稳定版本化的信封。状态 400、404、409、415、422 和 503 可以携带不同的 detail 形状。消费者必须根据操作、状态和记录的体型进行分类。

排除和网页曲面

六个 OpenAPI web 操作是 HTML 产品路线，而不是稳定的机器集成 API。
`/i18n/{locale}.json`、`/favicon.ico`、`/static/*`、生成的架构 界面 和以下划线前缀的 Python 帮助程序不包含在集成操作主题中。

健康反应限制

`/health` 和 `/health/ready` 仅公开紧凑状态字段。就绪情况不会报告每个依赖项的诊断、数据库详细信息、架构运行状况、文档门户状态或部署批准。操作监视器应将它们视为最小握手端点。

重用开发与运维编程接口示例和环境模板

对当前 BPM API 工作流程使用语言中立的请求模式、环境模板、可复制的 curl 命令、多部分示例和紧凑的 Python 断言。

为测试中的 BPM 实例设置 BPM_BASE_URL。请勿将凭据、租户名称、个人主机名、私有主机或生产配置文件 ID 放入可重用示例中；BPM 目前没有记录 API 身份验证。

此任务涵盖 API-HEALTH-001、API-HEALTH-002、API-PROFILE-001、API-PROFILE-003、API-PROFILE-004、API-PROFILE-005、API-FF-001、API-VAL-001 和 API-FF-002。这些示例有意使用 \$BPM_BASE_URL、合成名称、release-153、响应断言和一次性固定装置 ID，而不是特定于环境的主机或机密。

1. 在复制curl 请求之前创建可重用的环境模板。

请求示例：export BPM_BASE_URL="\${BPM_BASE_URL:?set BPM_BASE_URL first}"。响应示例：shell 为整个运行保留一个基本 URL、架构通道、合成配置文件名称和可选导入路径。错误示例：当缺少所需值时，shell 退出赋值。

```
# .env.devops-example.template
BPM_BASE_URL=
BPM_SCHEMA_CHANNEL=release-153
BPM_PROFILE_NAME=docs-api-example
BPM_JSON_IMPORT_PATH=./policies.json

export BPM_BASE_URL="${BPM_BASE_URL:?set BPM_BASE_URL first}"
export BPM_SCHEMA_CHANNEL="${BPM_SCHEMA_CHANNEL:-release-153}"
export BPM_PROFILE_NAME="${BPM_PROFILE_NAME:-docs-api-example}"
export BPM_JSON_IMPORT_PATH="${BPM_JSON_IMPORT_PATH:-./policies.json}"
```

2. 使用运行状况和就绪探测来控制每个可重用序列。

请求示例：curl -fsS "\$BPM_BASE_URL/health" 和 curl -fsS "\$BPM_BASE_URL/health/ready"。响应示例：{"status":"ok"} 和 {"status":"ready","ready":true}。错误示例：当使用 -f 时，curl 由于传输错误或非 2xx 响应而退出非零。

```
curl -fsS "$BPM_BASE_URL/health"
curl -fsS "$BPM_BASE_URL/health/ready"
```

3. 重用配置文件生命周期片段来创建、列出、读取和受保护的更新。

请求示例：POST \$BPM_BASE_URL/api/profiles、GET \$BPM_BASE_URL/api/profiles、GET \$BPM_BASE_URL/api/profiles/\$PROFILE_ID 和 PATCH \$BPM_BASE_URL/api/profiles/\$PROFILE_ID。响应示例：201 ProfileRead、200 ProfileRead[]、200 ProfileRead 和递增的 revision。错误示例：400、404、409 过时的 expected_revision 或 422。

```
PROFILE_JSON="$(curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-api-example","schema_version":"release-153","flags":{"DisableTelemetry":true}}' \
"$BPM_BASE_URL/api/profiles")"
```

```

PROFILE_ID="$(python -c 'import json,sys; print(json.load(sys.stdin)["id"])' <<<"$PROFILE_JSON")"
PROFILE_REVISION="$(python -c 'import json,sys; print(json.load(sys.stdin)["revision"])' <<<"$PROFILE_JSON")"

curl -fsS "$BPM_BASE_URL/api/profiles?q=docs-api-example&lifecycle=active&limit=50&offset=0"
curl -fsS "$BPM_BASE_URL/api/profiles/$PROFILE_ID"
curl -fsS -X PATCH -H 'Content-Type: application/json' \
-d '{"flags":{
  "DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":'$PROFILE_REVISION'}' \
"$BPM_BASE_URL/api/profiles/$PROFILE_ID"

```

4. 重用具有显式负载形状的验证和 JSON 导入片段。

请求示例：POST `$BPM_BASE_URL/api/validate/release-153` 和 POST

`$BPM_BASE_URL/api/profiles/import/firefox/policies.json`。响应示例：200

`{"ok":true,"profile":"release-153"}` 和 201 ProfileRead。错误示例：400、409、415、422 或 503。

```

curl -fsS -H 'Content-Type: application/json' \
-d '{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}' \
"$BPM_BASE_URL/api/validate/$BPM_SCHEMA_CHANNEL"

curl -fsS -H 'Content-Type: application/json' \
-d '{"name":"docs-json-import-example","schema_version":"release-153","compliance":{"source":"docs-devops-template"},"document":{"policies":{"DisableTelemetry":true}}}' \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"

```

5. 重用多部分导入和导出片段进行基于文件的切换。

请求示例：multipart/form-data 上传到 POST

`$BPM_BASE_URL/api/profiles/import/firefox/policies.json`，然后上传到 GET

`$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1`。响应示例：用于导入的 201 ProfileRead 和用于导出的包含规范 Firefox policies 的 200 application/json。错误示例：404 缺少配置文件或 422 无效配置文件 ID。

```

curl -fsS \
-F "name=docs-multipart-import-example" \
-F "schema_version=$BPM_SCHEMA_CHANNEL" \
-F 'compliance={"source":"docs-devops-template"}' \
-F "file=@$BPM_JSON_IMPORT_PATH;type=application/json" \
"$BPM_BASE_URL/api/profiles/import/firefox/policies.json"

curl -fsS "$BPM_BASE_URL/api/export/profiles/$PROFILE_ID/firefox/policies.json?pretty=1"

```

6. 当集成需要可重用响应断言时，使用紧凑的 Python 帮助程序。

请求示例：`session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)`。响应示例：除非状态和 JSON 正文与断言匹配，否则助手会立即引发。错误示例：集成产品必须记录断言失败、超时异常、过时的 `expected_revision` 或结构化 API 错误。

```
import os
```



```

import requests

base_url = os.environ["BPM_BASE_URL"].rstrip("/")
schema_channel = os.environ.get("BPM_SCHEMA_CHANNEL", "release-153")
session = requests.Session()

ready = session.get(f"{base_url}/health/ready", timeout=10)
ready.raise_for_status()
assert ready.json() == {"status": "ready", "ready": True}

payload = {"document": {"policies": {"DisableTelemetry": True, "BlockAboutConfig": True}}}
validation = session.post(f"{base_url}/api/validate/{schema_channel}", json=payload, timeout=10)
validation.raise_for_status()
assert validation.json() == {"ok": True, "profile": schema_channel}

created = session.post(
    f"{base_url}/api/profiles",
    json={"name": "docs-python-example", "schema_version": schema_channel, "flags":
payload["document"]["policies"]},
    timeout=10,
)
created.raise_for_status()
profile = created.json()
assert profile["schema_version"] == schema_channel

exported = session.get(
    f"{base_url}/api/export/profiles/{profile['id']}/firefox/policies.json?pretty=1",
    timeout=10,
)
exported.raise_for_status()
assert exported.json() == {"policies": payload["document"]["policies"]}

```

警告：

将响应状态、配置文件 ID、修订、模式通道、导出的文档哈希和断言结果保留在外部证据中。请勿将真实主机名、机密、API 密钥、机密值或生产标识符复制到文档示例中。

集成产品具有可重用的curl 和Python 示例，这些示例可以安全复制、独立于部署主机、与当前BPM API 行为同步，并与显式响应断言相关联。

在构建完整的外部控制产品工作流程时，将这些示例与拉取/列出/读取、应用前验证、导入-审核-导出和失败审核操作手册结合起来。

同步配置档案生命周期数据

使用配置文件列表、统计数据、读取、创建和更新端点，而不假设批量或事务行为。

设置 `$BPM_BASE_URL` 并读取当前 `ProfileCreate`、`ProfileUpdate` 和 `ProfileRead` 形状的 `/openapi.json`。

此任务涵盖 `API-PROFILE-001`、`API-PROFILE-002`、`API-PROFILE-003`、`API-PROFILE-004` 和 `API-PROFILE-005`。存储返回的 `id` 和 `revision`；不要仅从名字推断身份。

1. 在选择要同步的配置文件之前，使用过滤器列出配置文件。

请求示例：GET `$BPM_BASE_URL/api/profiles?`

`q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`。响应示例：200 `ProfileRead[]`。错误示例：422 表示无效的键入或范围查询输入。

2. 当您需要计数时，读取同一过滤器上下文的配置文件统计信息。

请求示例：GET `$BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`。响应示例：200 `{"filtered":1,"total":3}`。错误示例：422 表示查询输入无效。

3. 按整数 ID 读取所选配置文件，并仅在需要时包含存档记录。

请求示例：GET `$BPM_BASE_URL/api/profiles/42?include_deleted=false`。响应示例：带有 `id`、`revision`、时间戳、生命周期和验证状态的 200 `ProfileRead`。错误示例：404 表示配置文件丢失或不可见；422 表示无效路径或查询输入。

4. 仅当集成拥有新配置文件名称时才创建规范化 BPM 配置文件。

请求示例：带有 JSON `{"name":"corp-baseline","schema_version":"release-153","flags":{},"compliance":{}}` 的 POST `$BPM_BASE_URL/api/profiles`。响应示例：201 `ProfileRead`。错误示例：400 无效架构/配置文件、409 重复名称或 422 请求或策略验证。

5. 使用 PATCH 更新可变字段，并在需要乐观并发时发送 `expected_revision`。

请求示例：带有 JSON `{"description":"Reviewed by control plane","expected_revision":7}` 的 PATCH `$BPM_BASE_URL/api/profiles/42`。响应示例：200 `ProfileRead` 在材料更新后增加修订版。错误示例：400 架构/配置文件无效、404 丢失或已存档配置文件、409 过时的 `expected_revision` 或 422 验证错误。

6. 记录集成产品中的同步配置文件 ID、修订、验证状态、架构通道以及任何不透明的合规性元数据。

警告：

BPM 不提供多操作事务。如果后续步骤失败，集成产品将拥有恢复、审核记录和重试决策。

集成具有用于列表、统计、读取、创建和更新操作的可重复配置文件同步路径。

在归档、恢复、永久删除或重置操作之前使用破坏性生命周期任务。

存档、恢复、永久删除或重置配置档案

使用具有明确审查和恢复记录的破坏性配置文件生命周期端点。

在调用破坏性端点之前确认目标配置文件 ID、当前生命周期状态和外部批准记录。

此任务涵盖 API-PROFILE-006、API-PROFILE-007、API-PROFILE-008 和 API-PROFILE-009。BPM 没有批量确认令牌，没有永久删除的撤消端点，也没有跨操作的事务回滚。

1. 当您需要将配置文件从活动工作流程中删除同时保留以供审核时，请将其存档。

警告：

存档对于活动工作流程来说是一种破坏性的生命周期更改。在通话前记录批准原因和个人资料ID。

请求示例：DELETE \$BPM_BASE_URL/api/profiles/42。响应示例：没有 JSON 正文的 204。错误示例：404 缺少配置文件或 422 路径输入无效。

2. 仅在检查名称和配置文件用途是否仍然有效后才恢复存档的配置文件。

请求示例：POST \$BPM_BASE_URL/api/profiles/42/restore。响应示例：200 ProfileRead。错误示例：无法恢复或找到404个人资料； 422 路径输入无效。

3. 仅当外部保留策略允许删除时才永久删除配置文件。

警告：

永久删除会删除活动或存档的配置文件，并且没有 BPM 撤消端点。在通话前导出或记录所需的证据。

请求示例：DELETE \$BPM_BASE_URL/api/profiles/42/hard。响应示例：没有 JSON 正文的 204。错误示例：404 缺少配置文件或 422 路径输入无效。

4. 仅在一次性或明确批准的环境中重置库。

警告：

重置会永久删除库中的每个配置文件，并且当前不需要确认令牌。切勿将其用作例行同步快捷方式。

请求示例：DELETE \$BPM_BASE_URL/api/profiles/reset。响应示例：200 {"deleted":3}。错误行为：当前没有声明应用程序错误；数据库故障使用框架处理。

5. 在任何破坏性调用之后，重新列出配置文件并在适用时读取受影响的记录。

请求示例：GET \$BPM_BASE_URL/api/profiles?lifecycle=all。响应示例：200 ProfileRead[]。错误示例：422 查询输入无效。使用此验证来更新集成产品的库存状态。

6. 使用操作 ID、HTTP 方法、路径、配置文件 ID、批准者、原因、时间戳、响应状态和恢复决策写入外部审核记录。

预期结果：稍后的审阅者可以区分存档、恢复、硬删除和重置操作，并可以了解为什么集成认为操作安全。

破坏性配置文件生命周期操作记录有请求、响应、错误、警告和恢复证据。

如果审批记录丢失，请在调用永久删除或重置之前停止集成工作流程。

通过编程接口导入火狐浏览器策略文件

使用 JSON 或多部分输入从规范的 Firefox Enterprise policies.json 文档创建规范化的 BPM 配置文件。

设置 \$BPM_BASE_URL，选择受支持的架构通道（例如 release-153），并使用顶级 policies 对象准备完整的 Firefox 文档。

此任务涵盖 API-FF-001。导入接受带有 FirefoxPoliciesJsonImportRequest 的 application/json 或带有 file 字段的 multipart/form-data。BPM 验证规范的 Firefox 文档，将 policies 规范化为配置文件 flags，并将可选的不透明 compliance 元数据存储在 BPM 配置文件上。

1. 当集成产品内存中已有文档时，提交 JSON 导入请求。

请求示例：带有 Content-Type: application/json 和 JSON {"name": "api-json-import", "description": "Imported by control product", "schema_version": "release-153", "compliance": {"framework": "cis", "layer": "cis_l1"}, "document": {"policies": {"DisableTelemetry": true, "BlockAboutConfig": true}}}} 的 POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json。

2. 当源系统移交文件时使用分段导入。

请求示例：带有 multipart/form-data 的 POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json、字段 name=api-multipart-import、schema_version=release-153、可选的 description、可选的 JSON 字符串形式的 compliance={"framework": "cis", "layer": "cis_l2"} 和 file=@policies.json;type=application/json。运行时还接受多部分 document 字段，但 OpenAPI 多部分架构当前声明 file；使用 file 作为新示例。

3. 从 201 ProfileRead 响应中读取创建的 BPM 配置文件。

响应示例：201 ProfileRead 与 schema_version="release-153"、flags.DisableTelemetry=true、flags.BlockAboutConfig=true、已存储的 compliance、id 和 revision。存储返回的配置文件 ID，而不是从配置文件名称推断身份。

4. 将导入错误视为配置文件创建失败，并且不假定部分配置文件状态。

错误示例：400 表示格式错误的 JSON、错误的文档形状或不受支持的架构通道；409 表示重复的配置文件名称；415 表示不支持的媒体类型；422 表示请求或 Firefox 策略验证错误。

5. 在集成代码中将规范的 Firefox 和规范化的 BPM 形状分开。

请求 document.policies 是 Firefox 部署边界。创建的配置文件的 flags 是 BPM 的标准化模型。不要将顶级 flags 对象发送到此导入端点。

6. 在集成产品中记录进口证据。

警告：

导入创建 BPM 配置文件，但不部署 Firefox 设置、证明运行时强制执行或提供幂等性密钥。外部记录源文档哈希、架构通道、返回的配置文件 ID、修订、合规性元数据和重复名称恢复决策。

Firefox policies.json 文档表示为标准化 BPM 配置文件，可由 ID 审核、验证、导出和跟踪。

使用导出任务将保存的配置文件渲染回规范的 Firefox JSON，然后再将其交给部署工具。

通过编程接口导出规范火狐浏览器策略文件

将一个 BPM 配置文件呈现为 Firefox Enterprise 部署文档，并具有可选的下载和漂亮打印参数。

存储创建或导入返回的 BPM 配置文件 ID。如果配置文件已存档，请明确决定是否允许集成使用 `include_deleted=true` 导出它。

此任务涵盖 API-FF-002。导出读取 BPM 的规范化 flags 并返回带有顶级 policies 对象的规范 Firefox JSON。BPM 配置文件元数据和不透明的 compliance 元数据不会导出到 Firefox 部署文档中。

1. 将活动配置文件导出为规范的 Firefox JSON。

请求示例：GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json`。响应示例：带有 JSON `{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}` 的 200 application/json。

2. 当调用者需要附件文件名时添加下载模式。

请求示例：GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1`。响应示例：200 application/json 加 Content-Disposition: attachment; filename="profile-42-policies.json"。JSON 主体是相同的部署文档。

3. 有意选择格式化参数。

请求示例：当省略 indent 时，GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` 选择两个空格的缩进。请求示例：GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?indent=0` 使用显式缩进参数。错误示例：422 表示无效的约束查询值。

4. 仅当外部工作流程有审核原因时才导出存档的配置文件。

请求示例：GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?include_deleted=true&download=1&pretty=1`。响应示例：200 application/json。错误示例：404 当丢失或存档的配置文件不可见时；422 表示路径或查询输入无效。

5. 在将导出的文档交给另一个系统之前对其进行验证。

请求示例：带有 JSON `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}` 的 POST `$BPM_BASE_URL/api/validate/release-153`。响应示例：200 `{"ok":true,"profile":"release-153"}`。验证确认架构形状，而不是 Firefox 运行时部署是否成功。

6. 在外部记录准确导出的工件。

警告：

导出没有 ETag、前提条件标头、批量端点或事务包装器。记录配置文件 ID、导出前观察到的修订、架构通道、查询参数、导出的文档哈希以及 BPM 之外的下游切换决策。

该集成具有规范的 Firefox policies.json 工件，该工件排除仅限 BPM 的元数据，并且可以提供给部署工具。

如果验证失败或配置文件修订版在读取和导出之间发生更改，请停止切换并刷新配置文件状态。

通过编程接口验证候选火狐浏览器策略文件

使用验证端点作为候选 Firefox Enterprise policies.json 文档和结构化错误处理的预检门。

选择目标架构通道，例如 release-153 或 esr-140.13，并在 document.policies 中准备包含完整 Firefox 文档的 ValidationRequest 正文。

此任务涵盖 API-VAL-001。BPM 在导入、导出切换或外部策略审核门之前验证候选 policies.json 内容。为了兼容性，仍然接受简单的策略映射，但新的集成示例使用完整的 Firefox policies.json 形状。

1. 提交所选模式通道的成功验证请求。

请求示例：带有 JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}} 的 POST \$BPM_BASE_URL/api/validate/release-153。响应示例：200 {"ok":true,"profile":"release-153"}。

2. 即使 HTTP 状态为 200，也将 ok=false 视为失败的门。

请求示例：带有 JSON {"document":123} 的 POST \$BPM_BASE_URL/api/validate/release-153。响应示例：200 {"ok":false,"profile":"release-153","detail":"Expected object with policy mappings","error":"Expected object with policy mappings"}。检查响应正文，而不是单独检查 HTTP 状态。

3. 将格式错误的规范 Firefox 文档错误作为请求形状故障进行处理。

请求示例：带有 JSON {"document":{"policies":[]}} 的 POST \$BPM_BASE_URL/api/validate/release-153。错误示例：400，其中 detail.message="Firefox policies.json validation failed" 和 detail.issues[].path 指向 policies。

4. 在信任验证结果之前拒绝不受支持的模式通道。

请求示例：带有 JSON {"document":{"policies":{"DisableTelemetry":true}}} 的 POST \$BPM_BASE_URL/api/validate/beta-999。错误示例：带有详细信息文本 Unknown profile 'beta-999' 的 404。已注册但不可用的架构返回 503。

5. 将策略验证失败映射到可操作的审核者反馈。

请求示例：带有 JSON {"document":{"policies":{"Proxy":{"Mode":"bogus"}}}} 的 POST \$BPM_BASE_URL/api/validate/release-153。错误示例：带有 detail.message="Policy validation failed" 的 422 和问题字段 policy、path 和 message。

6. 在集成产品中记录验证决策。

警告：

验证只是一个模式和策略形状的门。它不部署 Firefox、证明运行时执行、验证调用者身份、保留配置文件修订或提供幂等密钥。在外部存储通道、候选文档哈希、响应状态、ok 值、结构化问题和审阅者决策。

集成可以阻止不安全的候选 policies.json 切换，并保留成功和失败决策的结构化证据。

如果验证成功，则使用相同的模式通道和记录的候选哈希继续导入、配置文件更新或导出切换。

运行生命周期 workflow 和从故障中恢复

运行拉取、比较和更新控制产品场景

使用现有配置文件和验证 API 来提取 BPM 状态，在外部进行比较，并使用显式所有权和并发证据更新一个配置文件。

设置 `$BPM_BASE_URL` 并决定哪个外部控制产品拥有比较逻辑、期望状态计算、重试策略和审计记录。BPM 不公开比较 API 或事务包装器。

此场景使用 API-PROFILE-001、API-PROFILE-003、API-PROFILE-005 和 API-VAL-001。BPM 拥有存储的配置文件状态和验证。集成产品拥有所需的状态、比较输出、审阅者批准和重试决策。

1. 使用控制产品用于所有权的相同过滤规则从库中提取候选配置文件。

请求示例：GET `$BPM_BASE_URL/api/profiles?q=corp&lifecycle=active&limit=50&offset=0`。响应示例：200 ProfileRead[]。错误示例：422 表示查询输入无效。

2. 通过 ID 读取每个选定的配置文件并存储其当前的 revision。

请求示例：GET `$BPM_BASE_URL/api/profiles/42`。响应示例：200 ProfileRead 与 flags、compliance、schema_version 和 revision。错误示例：404 缺少或已存档的配置文件；422 路径输入无效。

3. 将拉取的配置文件与 BPM 之外的外部期望状态进行比较。

预期结果：集成产品记录更改了策略密钥、未更改的密钥、丢失的密钥、审阅者和所需状态的来源。不要假设存在 BPM 比较端点、市场连接器或经过认证的工作流程。

4. 在应用更新之前验证候选 Firefox 文档。

请求示例：带有 JSON `{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}` 的 POST `$BPM_BASE_URL/api/validate/release-153`。响应示例：200 `{"ok":true,"profile":"release-153"}`。错误示例：400、404、422 或 503，具体取决于文档形状、渠道、策略有效性和架构可用性。

5. 仅使用比较之前观察到的修订来修补 BPM 配置文件。

请求示例：带有 JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}` 的 PATCH `$BPM_BASE_URL/api/profiles/42`。响应示例：带有递增的 revision 的 200 ProfileRead。错误示例：400、404、409 过时的 expected_revision 或 422。

6. 处理失败而不会自动重播。

警告：

如果验证失败、修订已过时或更新结果与预期状态不匹配，则停止工作流程，重新读取 BPM 状态并创建外部审核记录。BPM 不提供多操作事务、自动回滚、幂等密钥或安全重试合约。

控制产品具有可执行的拉取/比较/更新流程，具有明确的状态所有权、验证证据、修订保护和故障恢复边界。

如果更新的配置文件必须部署在其他地方，请继续执行 Firefox 导出任务并保留配置文件 ID、修订、验证响应和批准记录。

运行导入、审核、导出和合规性移交场景

使用现有的运行状况、导入、验证、配置文件和导出 API 来传递经过审核的 Firefox 文档以及合规性证据。

设置 \$BPM_BASE_URL，选择架构通道，并决定哪个系统拥有源文档、合规性扫描结果、审阅者批准、导出的工件和恢复决策。

此场景使用 API-HEALTH-001、API-HEALTH-002、API-FF-001、API-PROFILE-003、API-VAL-001 和 API-FF-002。BPM 存储规范化的配置文件和不透明的合规性元数据；集成产品拥有扫描解读和下游部署。

1. 在切换序列之前从健康和准备状态开始。

请求示例：GET \$BPM_BASE_URL/health 和 GET \$BPM_BASE_URL/health/ready。响应示例：200 {"status":"ok"} 和 200 {"status":"ready","ready":true}。错误示例：传输或网关故障在 BPM 外部处理，因为当前未声明应用程序错误。

2. 导入已审核的 Firefox 文档，并将合规性元数据附加到 BPM 配置文件。

请求示例：带有 JSON {"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","result":"passed","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}} 的 POST \$BPM_BASE_URL/api/profiles/import/firefox/policies.json。响应示例：201 ProfileRead。错误示例：400、409、415 或 422。

3. 读取创建的配置文件并与外部证据分开记录 BPM 所有权字段。

请求示例：GET \$BPM_BASE_URL/api/profiles/42。响应示例：200 ProfileRead 与 id、revision、flags、schema_version 和不透明的 compliance。错误示例：404 或 422。

4. 验证将要移交的规范 Firefox 文档。

请求示例：带有 JSON {"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}} 的 POST \$BPM_BASE_URL/api/validate/release-153。响应示例：200 {"ok":true,"profile":"release-153"}。错误示例：400、404、422 或 503。

5. 将 BPM 配置文件导出为规范的 Firefox 部署文档。

请求示例：GET \$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1。响应示例：200 application/json 与 Content-Disposition 和顶级 policies。BPM 配置文件元数据和合规性元数据不会导出。

6. 在不匹配时停止并写入故障恢复记录。

警告：

如果导入、验证、回读或导出证据与批准的源文档不同，请勿盲目重试。在 BPM 之外存储源哈希、配置文件 ID、修订、验证正文、导出哈希、扫描结果、审阅者、响应状态和恢复决策。

集成产品具有可执行的导入-审核-导出流程，具有合规性移交、规范导出和显式恢复证据。

使用 BPM 外部的部署工具来应用导出的 `policies.json`；BPM 验证并不能证明 Firefox 运行时强制执行。

从外部控制产品提取并检查浏览器策略管理器配置档案

构建具有显式所有权、过滤器和证据边界的控制产品集成的拉取/列表/读取端。

设置 `$BPM_BASE_URL`。决定哪个外部控制产品拥有库存选择、比对存储、重试策略和审核证据；BPM 仅拥有其 API 返回的配置文件记录。

此工作流程使用 API-PROFILE-001、API-PROFILE-002 和 API-PROFILE-003。它也是比较/更新、应用前验证、导入-审核-导出或合规性元数据移交工作流程之前的安全只读入口点。

1. 在读取 BPM 状态之前确认控制产品范围。

记录外部所有者、用途、模式通道、过滤规则以及是否排除存档的配置文件。使用 `lifecycle=active` 除非恢复过程明确需要 `include_deleted=true`。

2. 列出具有确定性过滤器和分页的配置文件。

请求示例：GET `$BPM_BASE_URL/api/profiles?`

`q=corp&lifecycle=active&limit=50&offset=0&sort=updated_at&order=desc`。响应示例：200 ProfileRead[]。错误示例：422 表示查询参数无效。

3. 从配置文件数据中单独捕获可选库存计数。

请求示例：GET `$BPM_BASE_URL/api/profiles/stats?q=corp&lifecycle=active`。响应示例：200 application/json。将计数证据存储为咨询库存上下文，而不是锁或事务边界。

4. 在比较之前读取 ID 所选择的每个配置文件。

请求示例：GET `$BPM_BASE_URL/api/profiles/42?include_deleted=false`。响应示例：200 ProfileRead 与 id、revision、schema_version、flags 和 compliance。错误示例：404 或 422。

5. 在 BPM 外部保留控制产品的读取快照。

存储配置文件 ID、修订、架构通道、过滤器查询、读取时间戳、源 URL、选定的策略密钥和控制产品运行 ID。不要在共享文档示例中存储机密或私有主机名。

6. 如果拉取结果不可信，则停止。

警告：

如果分页、过滤或回读证据不完整，请在更新之前停止。BPM 不会公开游标分页、快照隔离、比较锁、Webhooks 或用于读取工作流的安全重播协定。

外部控制产品具有可重现的拉取/列出/读取快照，并且知道哪些证据属于 BPM，哪些证据属于外部系统。

仅在外部系统记录了所选配置文件 ID 和修订后才继续应用前验证或比较/更新。

使用预期修订字段验证并应用配置档案更新

运行验证优先的更新工作流，该工作流使用当前修订版保护 BPM 状态，并在过时或无效数据时停止。

外部产品已读取目标配置文件并存储其当前的 revision。在发送任何配置文件补丁之前准备候选 Firefox policies.json 文档。

此工作流程使用 API-VAL-001、API-PROFILE-003 和 API-PROFILE-005。它记录了当前乐观的更新路径；BPM 不提供 ETags、前提条件标头、幂等性密钥或多操作事务。

1. 在 BPM 之外构建候选 Firefox 文档。

记录所需的策略密钥、源系统、审阅者、票证和模式通道。候选示例：`{"document":{"policies":{"DisableTelemetry":true,"BlockAboutConfig":true}}}`。

2. 在修补配置文件之前验证候选者。

请求示例：`POST $BPM_BASE_URL/api/validate/release-153`。响应示例：200 `{"ok":true,"profile":"release-153"}`。错误示例：400、404、422 或 503。

3. 当批准窗口很长时，在应用更新之前立即重新读取配置文件。

请求示例：`GET $BPM_BASE_URL/api/profiles/42`。响应示例：200 ProfileRead。如果 revision 发生更改，请重新运行比较和批准，而不是修补陈旧状态。

4. 仅用观察到的修订版修补可变字段。

请求示例：带有 JSON `{"flags":{"DisableTelemetry":true,"BlockAboutConfig":true},"expected_revision":7,"compliance":{"source":"control-product","decision":"approved"}}` 的 PATCH `$BPM_BASE_URL/api/profiles/42`。响应示例：带有递增的 revision 的 200 ProfileRead。

5. 将冲突响应视为停止信号。

错误示例：400 无效负载、404 缺少配置文件、409 过时的 expected_revision 和 422 架构级请求错误。409意味着必须重新计算外部期望状态。

6. 记录更新证据并避免盲目重放。

警告：

不要盲目地重试相同的 PATCH。在 BPM 之外存储旧修订版、新修订版、验证主体、补丁主体哈希、响应状态、审阅者批准和重试决策。

配置文件更新仅在验证和修订检查后应用，并具有成功或冲突的明确审计证据。

仅在读回配置文件与批准的所需状态匹配后导出配置文件或移交合规性元数据。

运行导入、审核、导出和合规性元数据移交

将外部审核或合规性产品连接到 BPM 导入和导出 API，而不将 BPM 元数据视为 Firefox 部署内容。

设置 `$BPM_BASE_URL` 并准备经过审核的 Firefox `policies.json` 源文档以及外部合规性元数据，例如扫描仪、票证、审核者和决策。

此工作流程使用 API-FF-001、API-PROFILE-003、API-VAL-001 和 API-FF-002。BPM 在配置文件上存储不透明的合规性元数据，但导出的 Firefox JSON 仅包含可部署的 `policies`。

1. 将审阅的源文档导入 BPM 配置文件中。

请求示例：带有 JSON `{"name":"reviewed-corp-baseline","schema_version":"release-153","compliance":{"scanner":"external-control","ticket":"SEC-42"},"document":{"policies":{"DisableTelemetry":true}}}` 的 POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json`。响应示例：201 ProfileRead。

2. 读回创建的配置文件并将 BPM 字段与外部证据分开。

请求示例：GET `$BPM_BASE_URL/api/profiles/42`。响应示例：200 ProfileRead 与 `revision`、`flags` 和不透明的 `compliance`。错误示例：404 或 422。

3. 在导出之前检查规范化策略集。

审核者检查导入的密钥、模式通道、源文档哈希、合规性元数据以及任何 BPM 规范化警告。BPM 不验证外部扫描仪结果或 Firefox 运行时强制执行。

4. 验证将导出的部署文档。

请求示例：带有 JSON `{"document":{"policies":{"DisableTelemetry":true}}}` 的 POST `$BPM_BASE_URL/api/validate/release-153`。响应示例：200 `{"ok":true}`。错误示例：400、404、422 或 503。

5. 导出规范的 Firefox 策略文档。

请求示例：GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?download=1&pretty=1`。响应示例：200 `application/json` 与 `Content-Disposition` 和顶级 `policies`。不导出合规性元数据。

6. 在下游部署之前写入切换记录。

警告：

在 BPM 之外存储源哈希、配置文件 ID、修订、验证响应、导出哈希、扫描结果、审阅者和下游所有者。不要将 BPM 导出视为 Firefox 已应用该策略的证据。

外部产品收到规范的 Firefox 部署文档和具有明确所有权边界的单独合规性移交记录。

使用 BPM 外部的部署工具将导出的 `policies.json` 放置在托管 Firefox 系统上。

通过浏览器策略管理器运行状况和就绪状态启动控制产品

在外部控制产品开始 API 同步之前，使用当前运行状况探测作为启动门。

BPM 源部署已启动。使用 `$BPM_BASE_URL`、超时、轮询间隔、最长等待时间以及门发生故障时要采取的操作来配置外部产品。

此工作流程使用 `API-HEALTH-001` 和 `API-HEALTH-002`。探头只是小的飞行前信号；它们不公开身份验证、授权、数据库诊断、模式缓存诊断、请求 ID 或服务级别目标。此操作手册集命名了健康门控启动、数据所有权、故障恢复以及证据过时不要盲目重试的规则。

文档助手的可用性独立于 BPM 核心的运行状况和就绪状态。在 0.9.3 中，每个助手问题都会显示已发布的本地模型训练提示；不会启动模型、检索、RAG 索引或外部来源请求。该边界不会阻止确定性文档搜索，也不得作为配置文件更改或 API 同步的门控条件。

1. 在打开同步队列之前检查基本活性。

请求示例：GET `$BPM_BASE_URL/health`。响应示例：200 `{"status":"ok"}`。传输错误由外部产品处理，因为没有为此探测器声明 BPM 应用程序错误主体。

2. 在配置文件或验证调用之前检查准备情况。

请求示例：GET `$BPM_BASE_URL/health/ready`。响应示例：200 `{"status":"ready","ready":true}`。如果 `ready` 不是 `true`，则保留队列并且不启动突变。

3. 记录启动门决策。

存储时间戳、基本 URL 标签、超时、探测状态、观察到的主体、控制产品版本和选择的下一步操作。不要将私有主机存储在共享操作手册示例中。

4. 在门通过后从只读库存开始。

在导入、导出、验证或修补操作之前调用 GET `$BPM_BASE_URL/api/profiles?limit=50`。这使得初始同步保持可观察和可恢复。

5. 仅在只读调用成功后才移至突变工作流程。

根据外部产品计划继续应用前验证、导入-审核-导出或导出工作流程。将重试和退避策略保留在 BPM 之外。

6. 当门不明确时关闭失败。

警告：

成功的探测不保留修订或保证以后 API 成功。失败或不明确的探测必须停止自动突变，直到操作员检查日志、源部署状态和网络/CORS边界。

仅当 BPM 响应当前运行状况和就绪握手时，外部控制产品才开始同步。

如果启动门失败，请使用管理员/DevOps 故障排除和源部署主题，而不是添加未记录的探测假设。

从集成失败中恢复并记录审计证据

处理验证、导入/导出、准备和修订失败，而无需不安全的重试或隐藏的事务假设。

当外部产品收到失败的 BPM API 响应、超时、意外的响应正文、过时的 `expected_revision` 或不匹配的导出证据时，请使用此工作流程。

此工作流连接 API-HEALTH-001、API-HEALTH-002、API-PROFILE-003、API-PROFILE-005、API-FF-001、API-FF-002 和 API-VAL-001。它定义了操作员证据；BPM 不提供分布式跟踪、自动回滚、批量事务恢复或 Webhook 通知。

1. 在重试之前对失败进行分类。

记录故障是否为运行状况/就绪、验证、导入、导出、读取、修补、冲突、超时或传输。保留 HTTP 状态，例如 400、404、409、415、422 或 503。

2. 冻结外部期望状态和 BPM 观察。

存储请求方法、路径、净化的有效负载哈希、配置文件 ID、观察到的 `revision`、模式通道、响应正文、响应时间和控制产品运行 ID。切勿在共享证据中包含秘密或私有主机名。

3. 当故障可恢复时，重新检查 BPM 运行状况并重新读取配置文件。

请求示例：GET `$BPM_BASE_URL/health`、GET `$BPM_BASE_URL/health/ready` 和 GET `$BPM_BASE_URL/api/profiles/42`。如果配置文件修订发生更改，请返回比较和批准。

4. 仅在操作员批准的决定后再次验证或导出。

为了进行验证，请使用相同的候选文档重复 POST `$BPM_BASE_URL/api/validate/release-153`。对于导出，重复 GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json?pretty=1` 并比较导出的文档哈希。

5. 停止对过时的修订或不明确的证据进行突变重试。

如果 `expected_revision` 返回 409，如果导出哈希不同，或者如果验证错误发生更改，则不要继续旧的运行。打开新的外部运行，进行新的阅读、比较和审阅者批准。

6. 将审核记录发布到外部记录系统。

警告：

BPM 存储配置文件状态，但不存储完整的集成审核跟踪。外部产品必须记录所有者、决策、票证、重试计数、停止原因、新旧修订、哈希值和操作员注释。

故障只能通过明确的证据、新的读取和操作员批准的重试决策来恢复，而不是通过盲目重放。

恢复后，将审核记录链接到成功的配置文件修订或将集成运行标记为已停止。

排除故障并评估生产就绪情况

对启动失败和无法访问的探测器进行故障排除

诊断未启动或未响应运行状况和就绪探测的源运行 BPM 流程。

当 `make dev`、`uvicorn app.main:app --reload --port 8000`、`GET /health` 或 `GET /health/ready` 在源部署或集成启动期间失败时，请使用此过程。

此诊断将过程、环境和探测证据分开。它不会创建 `systemd` 单元、Windows 服务、反向代理、TLS 端点或生产重启策略。

1. 捕获失败的命令和前台日志。

运行或复制确切的命令，然后从前台终端记录 `stdout` 和 `stderr`。包括 `make dev`、`uvicorn app.main:app --reload --port 8000`、Python 回溯摘要、工作目录和来自 `git rev-parse --short HEAD` 的源版本。

2. 在更改代码之前验证配置的主机和端口。

检查 `BPM_HOST`、`BPM_PORT`、`BPM_RELOAD`、`BPM_DATABASE_URL` 和 `BPM_DOCUMENTATION_SITE_DIR`。重点重播：`curl -fsS http://127.0.0.1:8000/health` 和 `curl -fsS http://127.0.0.1:8000/health/ready`。

3. 将进程启动与探针可达性分开。

如果进程未运行，请首先修复依赖项、环境或迁移证据。如果进程正在运行但探测无法访问，请检查本地防火墙、绑定地址、WSL 端口转发以及请求是否针对 `127.0.0.1`、`0.0.0.0` 或 Windows 主机路由。

4. 在收集证据时保留用户数据。

仅复制日志、环境名称和探测响应。不要将生成的工件、数据库行、架构缓存文件或源代码作为例行恢复进行编辑。不要删除 `data/bpm.db` 来使启动通过。

5. 选择最小的重新运行和升级路径。

重点重新运行：用于产品烟雾，仅当涉及文档输出时，而仅用于文档合同。

警告：

如果在配置和流程检查后探测器仍然无法访问，请停止自动集成运行并使用命令、日志、配置名称和探测器证据进行升级。

操作员可以区分失败的流程、无法访问的探针和部署边界，而无需更改用户数据。

探针通过后，在运行变异集成之前返回健康门控启动或可重用 API 示例主题。

排查架构缓存和编程接口验证错误

诊断不可用的模式缓存、未知的模式通道和 Firefox 策略验证失败，而无需修补生成的模式工件。

当 `POST $BPM_BASE_URL/api/validate/release-153`、`POST $BPM_BASE_URL/api/validate/beta-999` 或面向用户的验证返回 400、404、422 或 503 时使用此过程；非对象文档可以返回 200 `ok=false` 和 `Expected object with policy mappings`。

BPM 验证取决于维护的模式通道和本地模式/缓存配置。诊断记录问题是否是请求形状、未知配置文件、无效的 Firefox 策略数据或不可用的架构缓存。

1. 记录准确的验证请求和响应。

存储清理后的请求正文哈希、模式通道、HTTP 状态、`detail`、`message`、`error` 和 `issues`。如果候选 `policies.json` 包含客户值，请将其保留在日志之外。

2. 首先对请求形状故障进行分类。

非对象 `document` 或缺失 `document.policies` 是操作员输入，而不是模式缓存缺陷；当前的 BPM 可以将其报告为 200 `ok=false`。重点重播：`POST $BPM_BASE_URL/api/validate/release-153` 与 `{"document":{"policies":{"DisableTelemetry":true}}}`。

3. 对模式通道和缓存故障进行分类。

`beta-999` 等配置文件可以返回 404；不可用的注册模式可以返回 503。在进行更广泛的重建之前，请检查 `BPM_SCHEMA_BASE_URL`、`BPM_SCHEMA_CACHE_DIR` 和 `BPM_SCHEMA_HTTP_TIMEOUT`。

4. 使用维护的刷新和验证检查。

重点重新运行：用于 API 文档漂移或 API 行为更改时的产品架构测试。不要手动编辑 `app/schemas/cache` 或生成的 Firefox 框架。

5. 停止不安全的恢复并保存证据。

如果相同的有效请求在运行期间更改状态，则保留响应正文、源版本、缓存路径和环境变量。

警告：

不要将生成的模式工件或数据库行作为例行恢复进行编辑；将产品缺陷与操作员有效载荷错误分开。

运算符具有分类验证失败，重新运行次数最少且没有隐藏模式突变。

验证成功后，返回到“应用前验证”或“导入-审核-导出”工作流程。

火狐浏览器导入和导出失败故障排除

诊断 JSON 导入、多部分导入和 Firefox 策略导出失败，同时保留源和导出的证据。

当 POST `$BPM_BASE_URL/api/profiles/import/firefox/policies.json` 或 GET `$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` 返回 400、404、409、415 或 422 时，使用此过程。

导入/导出故障排除将源文档有效性、多部分字段形状、重复的配置文件名称、存档的配置文件访问以及规范的 Firefox 导出与下游部署工具分开。

1. 标识操作和内容类型。

记录请求是 JSON `FirefoxPoliciesJsonImportRequest`、带有 `file=@policies.json` 的 `multipart/form-data`，还是带有 `download=1`、`pretty=1` 或 `include_deleted=true` 的导出。

2. 在重试导入之前验证源 Firefox 文档。

重点重新运行：具有相同 `document.policies` 的 POST `$BPM_BASE_URL/api/validate/release-153`。如果验证失败，请在再次导入之前修复 BPM 之外的源文档。

3. 检查导入特定的运算符错误。

对于 409，选择新的合成配置文件名称或有意存档/恢复。对于 415，验证 `application/json` 或多部分上传字段。对于 422，记录字段级错误，不要直接重写配置文件存储。

4. 检查特定于导出的状态。

对于 404，重新读取 GET `$BPM_BASE_URL/api/profiles/42?include_deleted=true`。对于存档的配置文件，确定 `include_deleted=true` 是否合适。将导出的文档哈希值与审核后的源哈希值进行比较；BPM 配置文件元数据和合规性元数据不会导出。

5. 记录恢复和停止条件。

重点重新运行：用于记录示例。

警告：

导入/导出证据不匹配后不要盲目重试；存储源哈希、导出哈希、配置文件 ID、修订、HTTP 状态和操作员决策。

操作员可以识别故障是否属于源 JSON、多部分字段、配置文件生命周期状态或导出证据。

恢复后，将证据附加到外部控制产品审核记录中。

解决数据库和存储问题

诊断本地 SQLite、存储路径、备份和配置文件可见性问题，而无需直接编辑数据库行。

当配置文件消失、写入失败、迁移证据不清楚、备份校验和不匹配或 `BPM_DATABASE_URL` 指向意外位置时，请使用此过程。

当前源部署使用本地存储（例如 `sqlite+aiosqlite:///./data/bpm.db`）和 `data/` 下存储库拥有的文件、架构缓存路径和文档站点路径。BPM 中没有托管备份或恢复编排。

1. 确认活动数据库 URL 和工作目录。

记录 `BPM_DATABASE_URL`、当前目录、源版本以及进程是否从 Linux 文件系统或 `/mnt/c` 运行。不要假设两个 shell 指向同一个 `data/bpm.db`。

2. 在打开存储文件之前检查迁移和配置文件 API 证据。

重点重新运行：`alembic upgrade head`，然后是 `curl -fsS http://127.0.0.1:8000/api/profiles?limit=50`。如果 API 返回预期的配置文件，则症状可能是过滤器、生命周期或浏览器状态。

3. 在破坏性恢复之前验证备份证据。

检查 `mkdir -p backups`、`cp data/bpm.db backups/bpm-pre-update.db` 和 `sha256sum backups/bpm-pre-update.db` 记录。将关键的 `policies.json` 配置文件的导出保留为单独的证据。

4. 避免直接数据库行编辑。

请勿将 SQLite 行、生成的工件或架构缓存文件作为例行恢复进行编辑。请改用记录的配置文件存档、恢复、导入、导出或从源更新过程。

5. 当恢复不安全时使用存储证据升级。

包括数据库 URL、文件路径、校验和、迁移命令、API 响应和备份/导出证据。

警告：

如果不存在经过验证的故障前备份，则在删除或替换 `data/bpm.db` 之前停止；BPM 不提供时间点恢复。

操作员具有存储诊断功能，可保留用户数据并将数据库状态与 UI 或过滤器症状分开。

存储稳定后，重新运行运行状况/就绪状态和受影响最小的 API 工作流程。

子系统网络和过时依赖项故障排除

通过 WSL 访问问题、Linux 文件系统放置以及源更改后陈旧的 Python 依赖项诊断 Windows 10/11。

当 Windows 无法访问 WSL 托管的 BPM 进程、探针在 WSL 内部工作但不能从 Windows 工作、文件性能在 /mnt/c 下不稳定或依赖项在 `git checkout <approved-new-ref>` 后显得过时，请使用此过程。

WSL 故障排除将 Windows 浏览器访问、Linux 进程绑定、文件系统放置和 Python 环境刷新分开。BPM 在 0.9.1 中不提供本机 Windows 服务或安装程序。

1. 确认源签出和虚拟环境所在的位置。

更喜欢使用 Linux 文件系统进行签出和 `.venv`。记录文件是否在 /mnt/c 下，这可以改变性能和权限。诊断期间不要混合多次检查。

2. 验证 WSL 进程绑定和 Windows 路由。

检查 WSL 内的 `BPM_HOST="0.0.0.0"`、`BPM_PORT="8000"`、`curl -fsS http://127.0.0.1:8000/health` 以及从 Windows 进行的浏览器访问。如果 Windows 出现故障，请记录防火墙和端口转发证据。

3. 在源移动或修订更改后刷新依赖关系。

重点重新运行：`source .venv/bin/activate`、`python -m pip install --upgrade pip` 和 `pip install -e ".[dev]"`。如果依赖性错误仍然存在，请记录包名称和源版本，而不是手动编辑已安装的包。

4. 将过时的依赖症状与模式或数据库问题分开。

如果依赖项刷新后导入/导出或验证失败，请使用架构/缓存和导入/导出故障排除。如果配置文件可见性发生变化，请使用数据库/存储故障排除。

5. 停止不安全的本机 Windows 假设。

不要创建未记录的 Windows 服务、在文件系统之间移动数据库或声明本机安装程序行为。

警告：

如果 WSL 网络仍然不明确，请停止外部集成并使用 WSL 发行版、签出路径、绑定地址、Windows URL 和探测输出进行升级。

操作员可以在不更改产品存储的情况下区分 WSL 路由、文件系统放置和依赖项刷新缺陷。

WSL 访问稳定后，返回源部署验证和运行状况门控启动。

排查不可用的文档门户

当已安装的文档门户不可用时，收集 `/help/` 的可见状态和部署证据。

当已安装的 BPM 实例在 `/help/` 报告缺失、过期、不兼容、不完整或未找到状态时，使用此过程。

文档门户是 BPM 界面的一部分。其可用性取决于已安装的文档工件是否与正在运行的 BPM 版本匹配；BPM 不会在运行时修复缺失或不兼容的工件。先记录可见状态和部署信息，避免将门户问题误判为 API、浏览器缓存或产品配置问题。

1. 记录门户 URL 和可见状态。

记录完整的 `/help/` URL、区域设置、浏览器提示、BPM 版本和故障时间。将该响应与 FastAPI 的 `/docs` 和 `/openapi.json` 区分开来，它们是不同的界面。

2. 确认运行中的 BPM 实例及其文档设置。

记录进程主机、端口、源代码修订版本或已部署的发布标识，以及 `BPM_DOCUMENTATION_SITE_DIR` 的值；不要复制任何机密信息。

3. 状态未就绪时停止依赖文档的操作。

警告：

不要将缺失或不兼容的门户视为可用帮助系统，也不要手动编辑已安装文件来绕过该状态。

4. 将证据升级给 BPM 部署负责人。

提供门户 URL、可见状态、BPM 版本、文档站点设置名称、进程日志和观察时间。部署负责人可以通过受维护的文档过程恢复匹配的文档工件。

5. 恢复后验证门户。

部署负责人恢复匹配的工件后，重新打开 `/help/` 和受影响的上下文帮助路由。

操作员拥有足够证据，可将不可用的文档工件与 BPM API 区分开来，并请求正确的恢复操作。

将恢复后的门户状态、验证时间和原始升级证据一并记录，以便后续审计与交接。

查看网络暴露和代理准备情况问题

准备网络暴露说明和反向代理问题，而无需将不受支持的 TLS 或代理配方发布为 BPM 行为。

当前单节点源准备情况评估已完成，操作员知道用户是否将通过环回、WSL 转发地址、实验室主机或外部拥有的代理到达 BPM。

BPM 可以在配置的主机和端口上运行，但 BPM 文档不提供官方反向代理、TLS 终止、身份验证网关、标头策略或速率限制配方。

1. 选择满足当前评估的最小网络暴露。

更喜欢 127.0.0.1。仅当 WSL 转发或受控实验室网络需要时才使用 `BPM_HOST="0.0.0.0"`，并记录为什么 `BPM_PORT="8000"` 可达。

2. 在添加任何外部代理之前记下代理准备情况问题。

回答谁拥有 TLS 证书、主机名路由、请求大小限制、超时值、通过 `BPM_CORS_ALLOW_ORIGINS` 的 CORS 来源、转发标头、访问日志和事件响应。这些问题现在就可以准备；官方的 TLS/代理配方已计划但尚未实施。

3. 验证探头是否仍然可以通过所选边界工作。

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/health/ready
```

如果在实验室中使用外部拥有的代理，请记录直接源探测和代理端探测。请勿将代理 200 响应视为 BPM 身份验证、授权或 HA 的证明。

4. 明确标记不支持的网络声明。

警告：

BPM 不提供打包服务、官方 TLS/代理配方、支持的 HA 集群、WAF 规则、请求签名、速率限制、SSO、OAuth/OIDC、基于角色的访问控制或生产强化。将这些保留为外部控制或延期要求。

网络注释将现在可以配置的内容与仍然外部或延迟的反向代理和 TLS 决策分开。

在共享评估窗口启动之前使用监视、备份和更新窗口主题。

计划监控输入、备份证据和更新窗口

为源运行实例准备当前状态操作证据，同时避免有关托管监控、官方恢复自动化或滚动升级的声明。

实例已通过源就绪和网络暴露审核，并且单个管理员或 DevOps 所有者负责评估窗口。

在分布和运行时服务决策存在之前，运营规划应重点关注可观察的输入、手动备份和导出证据以及一个源节点的显式更新窗口。

1. 列出当前存在的监控输入。

记录进程状态、前台终端输出、stdout、stderr、GET /health、GET /health/ready、选定的 API 示例结果、通过 /help/ 访问文档以及来自 `git rev-parse --short HEAD` 的源修订。

2. 在窗口之前收集备份和导出证据。

```
mkdir -p backups
cp data/bpm.db backups/bpm-pre-update.db
sha256sum backups/bpm-pre-update.db
```

还导出代表性 Firefox 文档，例如 GET

`$BPM_BASE_URL/api/export/profiles/42/firefox/policies.json` 并存储生成的 `policies.json` 证据以及校验和。

3. 为源更改安排一个停止世界更新窗口。

对于 0.9.1，请在 `git fetch --tags`、依赖项刷新、`alembic upgrade head`、和 之前计划单节点停机时间。滚动升级已计划但尚未实施。

4. 将差距记录为阻碍因素，而不是隐藏的假设。

警告：

BPM 不提供托管监控、自动备份调度、官方恢复自动化、时间点恢复、托管机密、令牌轮换或滚动升级编排。使用更新操作手册中的手动证据和停止条件。

操作计划包含监控输入、备份/导出证据以及当前源操作的实际更新窗口。

将此证据附加到 HA 和延迟边界记录，以便以后的分发工作可以重用相同的清单。

浏览器策略管理器文档助手

在当前版本中使用文档助手

验证已发布的训练提示边界，而不使 BPM 或确定性文档搜索依赖聊天窗口。

使用受支持的 BPM 0.9.3 源码检出和 [浏览器策略管理器的最低系统要求](#) 中的基础资源。

在此版本中，聊天窗口仅是本地化的读者界面。它不会启动或安装模型、检索文档、使用 E5 索引、生成回答、显示引用或联系外部来源。确定性文档搜索保持独立且可用。

1. 使用正常的受支持源码流程启动 BPM。

不要为 0.9.3 的 BPM 启动添加模型、索引、嵌入或外部提供商配置。

2. 以读者的语言打开文档门户。

确认助手可从右下角打开，且普通搜索控件保持不变。

3. 提交一个与 BPM 有关的问题。

对话记录必须显示本地化提示：本地模型正在训练，该功能将在后续版本中提供。

4. 提交一个无关问题。

应显示相同提示；此版本的聊天窗口不是通用助手。

5. 使用确定性文档搜索查找当前答案。

搜索、筛选、导航和结果排序仍是受支持的产品信息路径。

助手显示已发布的本地化训练提示，BPM 和确定性搜索仍可独立使用。

不要为 0.9.3 安装模型或创建 RAG 生成。提示、语言或普通搜索边界不符合上述行为时，请记录文档缺陷。

维护当前版本的文档助手边界

在维护 BPM 文档时，保持本地化训练提示和确定性搜索边界的准确性。

准备已批准的 BPM 源码修订版和文档发布流程。

0.9.3 中没有需要维护的本地模型、模型工件、RAG 生成、嵌入存储、引用存储或外部来源提供商。请勿为本版本未交付的功能创建维护状态。

1. 应用已批准的 BPM 源码修订版并重新构建文档。

使用正常文档流程；准备开发环境时可使用 `make docs-install-dev`。

2. 在所有受支持语言中打开已发布文档。

确认助手入口仍可用，确定性搜索仍可独立工作。

3. 在每个语言区域提交一个问题。

确认显示相应语言的训练提示，并且没有回答、来源、引用、外部声明或模型安装流程。

4. 不要在 0.9.3 中运行模型安装或 RAG 生成命令。

这些命令属于后续版本，不能用于恢复训练提示。

5. 记录文档修订版和验证结果。

已发布提示与确定性搜索边界不一致时，应作为文档缺陷处理。

已发布门户在聊天窗口中保留本地化训练提示，并通过确定性搜索提供单独的当前信息路径。

不要将读者问题保留为维护工件。只有后续版本明确交付后，才重新评估模型和检索操作。